THE ENGINEER'S HANDBOOK

Authentication & Authorization

From first principles to system design. Identity, sessions, tokens, OAuth, passkeys, access control and distributed security — for web, mobile and backend engineers.

9 Phases | 45 Lessons | React · React Native · Node.js | System Design | Answer Guide

Edition: November 2026 · Aligned with NIST SP 800-63B-4, OWASP Top 10:2025 and RFC 9700

Contents

How to Use This Handbook 4

Phase 1 Foundations: Identity, Trust and the Primitives 7

1.1 Identity, Trust and the Three Questions 8
1.2 HTTP Is Stateless: How Systems Remember You 14
1.3 Credentials and Secrets 18
1.4 Cryptography for Engineers 22
1.5 Threat Modeling 25

Phase 2 Authentication Mechanics: Web and Mobile 27

2.1 Password Login Done Right 28
2.2 Server-Side Sessions 31
2.3 Cookies in Depth and CSRF 34
2.4 Tokens and JWTs 37
2.5 Sessions vs Tokens: The Real Trade-off 40
2.6 Token Storage in the Browser 43
2.7 Authentication on Mobile 45
2.8 The Account Lifecycle 48

Phase 3 Stronger Authentication 50

3.1 Multi-Factor Authentication 51
3.2 Passkeys, WebAuthn and FIDO2 54
3.3 Passwordless Flows 57
3.4 Adaptive and Risk-Based Authentication 59

Phase 4 Delegation and Federated Identity 61

4.1 The Delegation Problem 62
4.2 OAuth 2.0 Roles and Flows 64
4.3 PKCE and Public Clients 66
4.4 OpenID Connect 68
4.5 Scopes, Audiences and Consent 70
4.6 SAML and Enterprise SSO 72
4.7 OAuth Attacks and OAuth 2.1 74

Phase 5 Authorization Models 76

5.1 Authorization Fundamentals 77
5.2 Role-Based Access Control 80
5.3 Attribute-Based Access Control and Policy Engines 82
5.4 Relationship-Based Access Control 84
5.5 Multi-Tenant Authorization 86
5.6 Frontend vs Backend Authorization 88

Phase 6 System Design and Distributed Identity 90

6.1 Auth in Monoliths vs Microservices 91
6.2 Service-to-Service Identity 93
6.3 Zero Trust 95
6.4 Designing an Auth Service 97
6.5 Revocation, Caching and Consistency at Scale 99
6.6 System Design Challenges 101

Phase 7 Identity Management and the Enterprise 103

7.1 Identity Lifecycle and SCIM 104
7.2 Workforce IAM and Cloud IAM 106
7.3 API Keys and Secrets Management 108
7.4 Audit Logging and Compliance 110

Phase 8 The Industry: Build vs Buy 112

8.1 The Landscape 113
8.2 The Decision Framework 115

Phase 9 Breaking and Defending 117

9.1 Auth in the OWASP Top 10 and API Security Top 10 118
9.2 Testing Authentication and Authorization 120
9.3 Incident Response 121
9.4 Capstone 122

Appendix A: Answer Guide 123

Appendix B: Practical Projects Ladder 133

Appendix C: Security Review Checklist 134

Appendix D: Glossary 136

Appendix E: Further Reading 138

How to Use This Handbook

This handbook takes you from zero to an industry-ready understanding of authentication, authorization, identity management and application security. Its goal is not to make you memorize concepts or copy code. Its goal is to build the *engineering judgment* that lets you read unfamiliar implementations, find security flaws, make architectural decisions, and defend them in a design review.

THE ONE SENTENCE TO REMEMBER

A server receives bytes from an unknown source and must decide whether to act on them. Everything in this book is a tool for making that decision well.

The three viewpoints

Every concept is examined from three sides, because every real product has all three:

Party	Example	Do you control it?
Web client	React app running in a browser	**No.** The user owns the browser and can read and change everything.
Mobile client	React Native app on iOS/Android	**No.** The app bundle can be extracted, modified and its traffic intercepted.
Backend	Node.js API, databases, internal services	**Yes.** This is the only place security can be enforced.

The asymmetry in that last column is the single most important idea in the field. It returns in almost every lesson.

The running example: ShopSphere

SHOPSPHERE

ShopSphere is an e-commerce company with a **React web storefront**, a **React Native shopping app**, a **React admin dashboard** for staff, and a **Node.js API** backed by PostgreSQL and Redis. Customers buy things. Sellers manage their own products. Support staff can view orders but not refund them. Managers can issue refunds. Later, ShopSphere adds business customers who want to sign in with their company accounts, a partner API, and a microservice architecture.

Each lesson grows ShopSphere a little, so by the end you will have designed its entire identity system.

How every lesson is built

Each lesson follows the same structure so you always know where you are: objectives, the problem and why the concept exists, the deep explanation, real-world scenarios, diagrams and request flows, web and mobile perspectives, good versus bad decisions, attacks and failure modes, testing and debugging, implementation (from Phase 1.3 onward, code appears when it helps), industry practice, takeaways, and assessment scenarios.

Callout boxes are color-coded:

KEY IDEA

The insight to carry away. If you remember nothing else from a section, remember these.

FAILURE MODE

How things break in production, and how attackers exploit it.

WEB (REACT)

What changes when the client is a browser.

MOBILE (REACT NATIVE)

What changes when the client is a phone app.

INDUSTRY PRACTICE

What startups, enterprises and large tech companies actually do, using only publicly documented examples.

The study protocol

Reading is the smallest part of learning this material. For every lesson:

1. **Read it once for the big picture**, without stopping.
2. **Redraw every diagram from memory** on paper. If you cannot, re-read that section.
3. **Explain the key ideas out loud** as if teaching a junior developer. Wherever you stumble is what to review.
4. **Write answers to the assessment scenarios before checking Appendix A.** Your honest reasoning is the point. The answer guide lists what a strong answer covers, not a script to memorize.
5. **Do the practical task** where one is given. Phases 2 onward include hands-on work.
6. **Bring hard questions back to a conversation with Claude.** A PDF cannot evaluate your reasoning or answer follow-ups; a mentor can. Paste your answers and ask for a review.

Plan on roughly one lesson per day, or three to four per week if you are working full-time. The whole book is a three-to-four-month journey done properly. Rushing it defeats the purpose.

Standards this edition follows

This edition reflects guidance current as of late 2026, including **NIST SP 800-63B-4** (Digital Identity Guidelines, final July 2025), the **OWASP Top 10:2025**, **RFC 9700** (OAuth 2.0 Security Best Current Practice, January 2025), the **OAuth 2.1** consolidation effort, **RFC 8725** (JWT Best Current Practices), and the **WebAuthn/FIDO2** passkey standards. Libraries and vendors change fast; always verify current versions and APIs against official documentation before using them in production.

PHASE 1

Foundations: Identity, Trust and the Primitives

Before any login form or token, you need the mental models: what identity is, why the web forgets you, how secrets are stored, which cryptographic tools exist, and how attackers think. Every later lesson depends on these.

IN THIS PHASE

1. Identity, Trust and the Three Questions
2. HTTP Is Stateless: How Systems Remember You
3. Credentials and Secrets
4. Cryptography for Engineers
5. Threat Modeling

LESSON 1.1

Identity, Trust and the Three Questions

Why authentication and authorization exist, and the trust boundary that governs every system.

Learning objectives

- Explain *why* authentication and authorization exist, from the physical constraints of networked computing.
- Distinguish identification, authentication, authorization and auditing, and spot when someone conflates them.
- Understand that authentication produces **confidence**, never certainty.
- Identify the **trust boundary** in any web or mobile system and explain why clients can never be trusted.
- Recognize the most common real-world failure: treating "logged in" as "allowed."

The problem: computers can't see you

At a bank branch, the teller recognizes your face, checks your ID, compares your signature, and notices if you look nervous. Dozens of signals, many unconscious. Now put the bank online. What does the server receive?

```
POST /transfer HTTP/1.1
Host: api.bank.com
Content-Type: application/json

{ "from": "ACC-1001", "to": "ACC-9999", "amount": 50000 }
```

A sequence of bytes from somewhere. No face, no ID card. The server cannot tell whether this came from you on your phone, a script in another country, or your own browser being tricked by a malicious website.

> KEY IDEA
>
> **The root problem: a server receives bytes from an unknown source and must decide whether to act on them.**

The four questions

When a request arrives, a well-designed system answers four distinct questions. Most bugs come from collapsing two into one.

Identification: "Who do you claim to be?" An assertion: a username, an email, an account number. Claims are free. Anyone can type `admin@company.com`.

Authentication (AuthN): "Prove it." Gathering *evidence* that the claim is true. Evidence comes in **factors**: something you *know* (password), something you *have* (phone, hardware key), something you *are* (fingerprint). Evidence can be stolen, guessed or forged, so authentication gives a *level of confidence*, never certainty. Engineers choose how much confidence each action requires.

Authorization (AuthZ): "Are you allowed to do this?" A policy decision about a **principal** (who), an **action** (what verb), a **resource** (on what) and **context** (when, from where, how recently verified). Knowing you are Priya does not tell me whether Priya can read invoice #4471.

Auditing (Accounting): "What happened, and who did it?" The record that lets you investigate later. Together these are often called "AAA."

> **THE MENTAL MODEL THAT MATTERS MOST**
>
> **Authentication is an event. Authorization is a decision made on every request.**

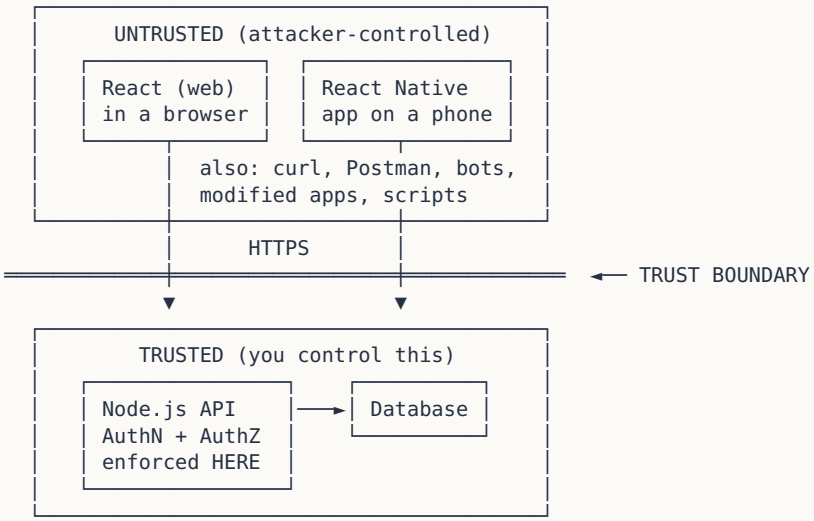
You authenticate once, and the system *remembers* it with a session or token. Authorization must be evaluated freshly for every action. Between login and a request three hours later, your password may have been reset, your role removed, your laptop stolen. The system is acting on an *old* proof. How old is acceptable is a design decision, and it sits at the heart of Phase 2.

Identity is not the same as a person

- **One person, many identities:** personal and work Google accounts.
- **Many people, one identity:** a shared family streaming account.
- **Non-human identities:** services, CI pipelines, cron jobs and devices. In modern systems they vastly outnumber humans and are a growing source of breaches (Phase 6).

The general term is **principal**: anything that can be authenticated and can act.

The trust boundary



Your own apps sit in the untrusted zone.

WEB (REACT)

Your React code is shipped to the user's browser. They can read every line in DevTools, change variables, skip `if (isAdmin)` checks, and send any request they like.

MOBILE (REACT NATIVE)

A compiled app feels protected but is not. The JS bundle can be extracted from an APK/IPA and read; traffic can be intercepted with mitmproxy or Proxyman. Any secret embedded in the app is public the day you ship. And an attacker does not need your app at all: your API is just a URL they can `curl`.

KEY IDEA

The client is a suggestion. The server is the law.

The hotel analogy

```
You arrive          Front desk           Keycard          Room door
"I'm Rahul"  ──►    checks your ID ──►   issued     ──►   reads card:
(identification)    (authentication)     (session/token)  "Room 412? ✓"
                                                          (authorization)
                    Door logs every entry   (auditing)
```

The door does not check your passport; it checks the *card*. That is exactly how sessions and tokens work, and it reveals the dangers ahead: whoever holds the card *is* you (a **bearer credential**); a lost card must be invalidated (the **revocation problem**); and the card opens room 412 and the gym but not room 413 (**authorization**).

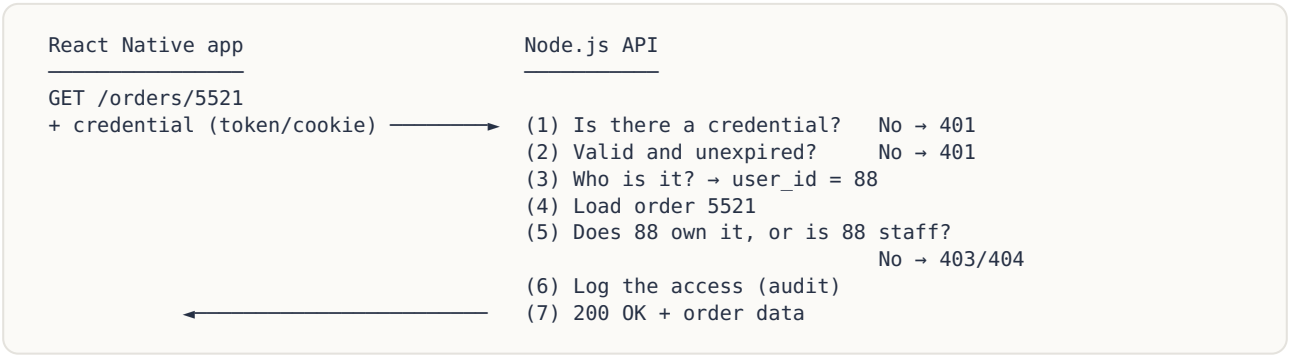
Real-world scenarios

Netflix profiles. The account is the authenticated identity. Profiles were long a preference, not a security boundary; anyone could tap "Dad." Netflix later added an optional profile PIN, a lightweight extra authentication step. Lesson: be precise about where your security boundary actually is.

Your banking app. Fingerprint unlocks your balance, but sending ₹2,00,000 to a new payee asks for an OTP. Different actions need different confidence. This is **step-up authentication** (Phase 3).

Google across products. Sign into Gmail, open YouTube and Drive, and you are already signed in: **single sign-on** (Phase 4). Authorization still differs per product.

The request flow



The web app sends the same request. **The backend must not care which client sent it.** Note: HTTP `401 Unauthorized` really means *unauthenticated*; `403 Forbidden` means "I know who you are, and the answer is no."

Good vs bad decisions

Bad	Consequence	Good
Any logged-in user can fetch any `/orders/:id`	Customers iterate IDs and read everyone's orders (IDOR/BOLA)	Check ownership or role for every resource
Server reads `userId` or `role` from the request body	Attacker changes the field and becomes someone else	Identity comes only from the verified credential
"Refund" button hidden in React for non-managers; endpoint unchecked	Support agent calls the endpoint with curl	UI gating for UX, server check for security
Credentials valid for 30 days with no revocation	Suspended fraudster stays logged in	Deliberate lifetime and revocation design

FAILURE MODE: BROKEN ACCESS CONTROL

In 2019, First American Financial was publicly reported to have exposed hundreds of millions of mortgage-related documents because document URLs used sequential IDs and anyone with one link could change the number. Broken Access Control is ranked **A01 in the OWASP Top 10:2025**, as it was in 2021.

Testing and debugging habits

- **Two-account test:** as User B, request every resource User A created. All must fail.
- **No-credential test:** strip the credential; expect `401`.
- **Tamper test:** change every ID, user field and role in the request.
- **Bypass-the-client test:** call the endpoint with curl. Any protection that disappears was never a protection.
- **Debug by status code:** `401` points to authentication (missing/expired credential, cookie not sent, clock skew); `403` points to the authorization policy.

The analysis framework

```
PRINCIPAL:  Who is making this request? How do we know?
ACTION:     What are they trying to do?
RESOURCE:   On what, specifically, and who owns it?
CONTEXT:    Time, device, network, how recently they authenticated?
DECISION:   Allow or deny, and WHERE is it enforced?
```

Takeaways

- Identification is a claim; authentication is evidence; authorization is a per-action policy decision; auditing is the record.
- Authentication gives confidence, never certainty, and confidence should match risk.
- Web and mobile clients are both untrusted. Code and secrets shipped to them are visible.
- Identity comes from verified credentials, never from client-supplied fields.
- "Logged in" never means "allowed."

Assessment

SCENARIO 1 · THE HIDDEN BUTTON

The React admin dashboard shows "Delete Seller" only when `currentUser.role === "admin"`. `DELETE /sellers/:id` only checks for a valid credential. A teammate says, "It's secure, non-admins never see the button." Respond, describe two concrete exploits (one through the dashboard, one with no ShopSphere client), and say exactly what must change and where.

SCENARIO 2 · THE SECRET IN THE APP

The PM proposes embedding a secret API key in the React Native app so "only our official app can call the API." What does this protect against, what does it fail to protect against, and is it authentication of a user, an app, or neither?

SCENARIO 3 · WHICH QUESTION FAILED?

Classify as AuthN or AuthZ failure: (a) credential stuffing with passwords leaked from another site; (b) a seller edits a competitor's product by changing the ID; (c) a support agent calls the refund endpoint directly; (d) a session credential stolen from a public computer.

SCENARIO 4 · 403 OR 404?

User 55 requests `GET /orders/9812`, owned by user 77, and gets `403`. A reviewer suggests `404`. Explain the reasoning, the downsides, your choice for orders, and whether it changes when *existence itself* is sensitive.

SCENARIO 5 · THE STALE PROOF

A customer changes a leaked password on the web. Their phone app stays logged in, and so does the attacker's session. Is this a bug? Explain the conceptual problem and what the system *should* do, with trade-offs.

LESSON 1.2

HTTP Is Stateless: How Systems Remember You

Every request arrives as a stranger. Cookies, headers and sessions are how we stitch them into a conversation.

Learning objectives

- Explain what "stateless" means for HTTP and why it was designed that way.
- Describe the two dominant ways clients carry identity on each request: **cookies** and the **Authorization header**.
- Understand how browsers and mobile apps differ in handling cookies.
- Recognize where "state" can live (client vs server) and the trade-offs of each.

The problem

HTTP was designed so that each request is self-contained. The server does not have to remember the previous request to answer the next one. This is why the web scales: any server in a fleet of a thousand can answer any request, and a server can crash and restart without "losing the conversation."

But logging in is inherently stateful. You prove yourself on request #1, and you expect requests #2 through #500 to be treated as *you*. Since the protocol forgets, **something must be carried on every request** to re-establish who you are. That "something" is the hotel keycard from Lesson 1.1.

Two ways to carry the keycard

1. Cookies (browser-managed)

The server tells the browser to store a value, and the browser automatically attaches it to future requests to that site.

```
HTTP/1.1 200 OK
Set-Cookie: sid=9f8a7c...; HttpOnly; Secure; SameSite=Lax; Path=/

-- later, automatically, on every request to the same site --
GET /orders HTTP/1.1
Cookie: sid=9f8a7c...
```

The crucial word is **automatically**. JavaScript does not need to do anything; the browser attaches it. That is convenient, and it is also the root of CSRF attacks (Lesson 2.3), because the browser attaches cookies even when *another* site triggers the request.

2. The Authorization header (application-managed)

The client stores the credential itself and attaches it explicitly:

```
GET /orders HTTP/1.1
Authorization: Bearer eyJhbGciOiJFZERTQSJ9...
```

Nothing is automatic. Your code decides when to send it. That avoids CSRF, but means your code must *store* the credential somewhere, which raises the question: where, and who else can read it? (Lessons 2.6 and 2.7.)

	Cookie	Authorization header
Who attaches it	Browser, automatically	Your code, explicitly
Readable by page JavaScript	Not if `HttpOnly`	Yes, it must be, to attach it
CSRF risk	Yes, needs mitigation	No
XSS theft risk	Low with `HttpOnly`	High if stored in JS-accessible storage
Natural fit	Browser apps on the same site	Mobile apps, APIs, service-to-service

WEB (REACT)

A React SPA on `shop.com` calling an API on `shop.com` (or a same-site subdomain) fits cookies naturally. Cross-origin APIs need CORS with `credentials: 'include'` and careful configuration. The industry trend for browser apps is to keep tokens *out* of JavaScript and use HttpOnly cookies, often through a Backend-for-Frontend.

MOBILE (REACT NATIVE)

A React Native app is not a browser. There is no "site," no same-origin policy, no automatic cross-site requests, so CSRF does not apply in the same way. Native networking stacks have cookie jars, but they behave inconsistently across iOS and Android. Most mobile apps therefore store a token in the OS secure storage (Keychain/Keystore) and send it in the `Authorization` header.

Where does state live?

There are two fundamental designs, and you will meet them again in Lesson 2.5:

```
SERVER-SIDE STATE (sessions)                CLIENT-SIDE STATE (self-contained tokens)
-----------------------------               -----------------------------------------
Client holds: random ID "9f8a7c"            Client holds: signed data
                                                 {user: 88, role: "seller", exp: ...}
Server holds: 9f8a7c → {user: 88, ...}      Server holds: only the key to verify it

+ instant revocation (delete the row)       + no lookup, scales across services
+ small cookie                              - hard to revoke before expiry
- needs a shared store (Redis)              - data can be stale (role changed?)
```

KEY IDEA

Statelessness is a property of the protocol, not of your system. Every authenticated system keeps state somewhere. The design question is *where*, and what you lose by putting it there.

Failure modes

- **Credential in the URL** (`/orders?token=abc`): URLs leak into server logs, browser history, analytics, and the `Referer` header sent to third parties. Never put long-lived credentials in URLs.
- **Assuming the cookie is the user.** A cookie is just a value the client sends. The server must validate it on every request.
- **Mixing mechanisms inconsistently**, such as accepting both a cookie and a header on the same endpoint without understanding that the cookie path is CSRF-exposed.

Testing and debugging

- In DevTools, open **Application → Cookies** to see what is stored and with which flags; open **Network** to see what is actually sent.
- Common "I keep getting logged out" causes: cookie set for the wrong `Domain` / `Path` , `Secure` cookie on an `http://` dev server, `SameSite` blocking a cross-site request, or a missing `credentials: 'include'` on `fetch` .
- For React Native, use an intercepting proxy to confirm the `Authorization` header is actually present.

Takeaways

- HTTP forgets; your system must carry identity on every request.
- Cookies are attached automatically by browsers (convenient, CSRF-prone). Headers are attached by your code (explicit, storage-sensitive).
- Mobile apps usually use headers; browser apps increasingly use HttpOnly cookies.
- State always lives somewhere. Choosing where is a trade-off between revocation and scale.

Assessment

SCENARIO 1 · THE MAGIC LINK IN THE URL

A teammate proposes keeping users logged in by appending `?session=<id>` to every link in the web app, "so it works even with cookies disabled." List every place that session ID could leak, and propose an alternative.

SCENARIO 2 · ONE API, TWO CLIENTS

ShopSphere's API serves both the React site and the React Native app. Propose how each client carries identity, and explain why using the same mechanism for both might *not* be ideal.

SCENARIO 3 · LOGGED OUT ON REFRESH

Login works on the deployed site, but in local development (`http://localhost:3000` frontend, `http://localhost:4000` API) users appear logged out on every request. Name three plausible causes related to this lesson.

LESSON 1.3

Credentials and Secrets

How to store something you must verify but must never be able to read.

Learning objectives

- Explain why passwords must be **hashed**, not encrypted.
- Understand salts, slow hashing and why bcrypt, scrypt and Argon2 exist.
- Know what current guidance (NIST SP 800-63B-4) says about password rules.
- Recognize timing leaks and other subtle credential-handling bugs.

The problem

To verify a password, the server must compare what the user typed with something it stored. If it stores the password itself, then anyone who reads the database (an attacker, a rogue employee, a leaked backup) gets every password. And because people reuse passwords, a leak from your site unlocks their email and bank accounts too. Your database will eventually leak; design as if it already has.

Hashing vs encryption

Encryption is reversible: with the key you get the original back. If you encrypt passwords, the key lives somewhere near the database, and an attacker who gets one usually gets both.

Hashing is one-way: `hash(password)` produces a fingerprint, and there is no "unhash." To verify, you hash what the user typed and compare fingerprints. You never need the original password, so you should never be able to recover it.

> KEY IDEA
>
> **If you can email a user their forgotten password, your system is broken. Passwords should be verifiable, never recoverable.**

Why plain SHA-256 is not enough

General-purpose hashes like SHA-256 are designed to be *fast*. That is a feature for checksums and a disaster for passwords: a single modern GPU can compute billions of SHA-256 hashes per second, so an attacker can try an entire dictionary against your leaked table in minutes.

Two further problems: identical passwords produce identical hashes (revealing who shares a password), and attackers can precompute hashes of common passwords (**rainbow tables**).

The fixes:

- **Salt:** a unique random value per user, stored next to the hash and mixed in before hashing. Identical passwords now produce different hashes, and precomputation is useless.

- **Slow, memory-hard hashing:** algorithms *deliberately* expensive to compute, with a tunable cost. **Argon2id** (the current recommendation of the OWASP Password Storage Cheat Sheet), **scrypt**, and **bcrypt** (older, still acceptable, but it truncates input at 72 bytes). Memory-hardness makes GPU and custom-hardware attacks far more expensive.

```
Registration:  password ──► Argon2id(password, random salt, cost) ──► store:
               $argon2id$v=19$m=19456,t=2,p=1$<salt>$<hash>

Login:         typed pw ──► Argon2id(typed, SAME salt, SAME params)
                        ──► compare with stored hash (constant time)
```

The encoded string stores the algorithm, parameters and salt together, so you can raise the cost later and rehash users on their next successful login.

Implementation (Node.js)

```
import argon2 from "argon2";

// Registration
export async function hashPassword(plain) {
  return argon2.hash(plain, { type: argon2.argon2id }); // salt generated for you
}

// Login
export async function verifyPassword(storedHash, plain) {
  const ok = await argon2.verify(storedHash, plain);
  if (ok && argon2.needsRehash(storedHash)) {
    // parameters were raised since this hash was created: upgrade it now
  }
  return ok;
}
```

What NIST SP 800-63B-4 (2025) says about password policy

The 2025 revision overturns much "common wisdom":

- **Length beats complexity.** Passwords used as the only factor should be at least **15 characters**; at least 8 when used with MFA. Allow at least 64 characters.
- **No composition rules** ("one uppercase, one symbol"). They make passwords harder to remember, not harder to crack.
- **No forced periodic rotation.** Require a change only on evidence of compromise.
- **Check against blocklists** of common and breached passwords (for example via Have I Been Pwned's k-anonymity API).
- **Allow paste and password managers.** Blocking paste pushes users to weaker passwords.

FAILURE MODES

Logging request bodies that contain passwords. Comparing secrets with `==` (which can leak timing). Returning different errors for "no such user" vs "wrong password" (user enumeration). Hashing on the *client* and sending the hash, which simply makes the hash the new password. Rolling your own scheme.

Timing-safe comparison. When comparing secrets you generated (API keys, reset tokens), use `crypto.timingSafeEqual` on equal-length buffers, or better, compare *hashes* of the secret, so that response time does not reveal how many leading characters matched.

Not all secrets are passwords

Secret	Who chooses it	Storage
User password	Human (low entropy)	Slow hash: Argon2id/bcrypt
API key, reset token, refresh token	Your system, 128+ random bits	Fast hash (SHA-256) is fine, because the input is already unguessable
Signing key, DB password	Your system	A secrets manager or KMS, never the code repository

Generate random secrets with a CSPRNG: `crypto.randomBytes(32).toString("base64url")`. Never `Math.random()`.

WEB (REACT)

Never hash passwords in the browser as a "security" measure. Rely on HTTPS to protect the password in transit and hash on the server. Do use `autocomplete="current-password"` / `"new-password"` so password managers work.

MOBILE (REACT NATIVE)

Do not store the user's password on the device "for auto-login." Store a revocable token in secure storage instead (Lesson 2.7). Support iOS/Android password autofill with the correct `textContentType` / `autoComplete` props.

Takeaways

- Hash passwords with a slow, salted, memory-hard function (Argon2id preferred); never encrypt or store them.
- Machine-generated secrets need randomness, not slowness; hash them before storing.
- Modern policy: long passwords, blocklists, no composition rules, no forced rotation.

Assessment

SCENARIO 1 · THE LEGACY TABLE

You join a company whose `users` table stores unsalted MD5 hashes for 2 million users. You cannot ask everyone to reset their password. Design a migration that upgrades all accounts to Argon2id without waiting for users to log in.

SCENARIO 2 · THE 'SECURE' FRONTEND HASH

A teammate hashes passwords with SHA-256 in the React app before sending them, "so the password never leaves the device." Explain what this protects, what it does not, and what the stored value on the server effectively becomes.

SCENARIO 3 · RAISING THE COST

Security asks you to triple your Argon2 cost parameters. Login latency goes from 80 ms to 240 ms and CPU usage rises sharply during peak traffic. What are the trade-offs, and how do you protect the login endpoint from becoming a denial-of-service target?

LESSON 1.4

Cryptography for Engineers

The five tools you will use constantly, what each guarantees, and how they are misused.

Learning objectives

- Distinguish hashing, HMAC, symmetric encryption, asymmetric encryption and digital signatures by *what they guarantee*.
- Explain what TLS does and does not protect.
- Choose the right primitive for common auth tasks.

The problem

Authentication systems are built from cryptographic primitives. You do not need to implement them (and must not), but you need to know which guarantee each one provides. Most crypto bugs in application code are not broken algorithms; they are the *wrong tool* for the job.

The toolbox, by guarantee

Tool	Guarantee	Key	Auth example
Hash (SHA-256)	Integrity fingerprint; anyone can compute it	None	Storing API keys, PKCE challenge
HMAC (HMAC-SHA256)	Integrity + authenticity: only key holders could have produced it	One shared secret	Signed cookies, webhook signatures, HS256 JWTs
Symmetric encryption (AES-GCM)	Confidentiality (+ integrity with AEAD)	One shared secret	Encrypting session data, data at rest
Asymmetric encryption (RSA-OAEP, ECIES)	Confidentiality to the key owner	Public encrypts, private decrypts	Key exchange; rare in app code
Digital signature (Ed25519, ECDSA, RSA-PSS)	Authenticity + integrity, verifiable by anyone with the public key	Private signs, public verifies	RS256/ES256/EdDSA JWTs, passkeys, OIDC

KEY IDEA

Encoding is not encryption. Encryption is not integrity. Signing is not encryption. A JWT is *signed*, so anyone can read its contents; it is only protected from *modification*.

HMAC vs signatures: the architectural difference

```
HMAC (symmetric)                          Signature (asymmetric)
─────────────────                         ──────────────────────
Auth server ── shared secret ── API A     Auth server holds PRIVATE key
            └── shared secret ── API B    APIs hold only the PUBLIC key

Any service that can VERIFY can also      Services can verify but can never
FORGE. One leak compromises everything.   forge. Public keys can be published
                                          (JWKS) and rotated.
```

This is why distributed systems and identity providers (Google, Microsoft, Auth0) sign tokens asymmetrically and publish their public keys at a JWKS endpoint (Lesson 6.4).

Implementation (Node.js `crypto`)

```
import crypto from "node:crypto";

// HMAC: verify a webhook signature
function verifyWebhook(rawBody, headerSig, secret) {
  const expected = crypto.createHmac("sha256", secret).update(rawBody).digest();
  const given = Buffer.from(headerSig, "hex");
  return given.length === expected.length && crypto.timingSafeEqual(given, expected);
}

// Ed25519 signature
const { publicKey, privateKey } = crypto.generateKeyPairSync("ed25519");
const sig = crypto.sign(null, Buffer.from("user=88"), privateKey);
const valid = crypto.verify(null, Buffer.from("user=88"), publicKey, sig); // true
```

Note that the webhook check hashes the **raw** body. Re-serializing parsed JSON changes bytes and breaks verification, a very common production bug.

TLS: what it does and does not do

TLS (the "S" in HTTPS) gives you confidentiality and integrity *in transit*, and proves to the client that it is talking to the real server (via certificates). It does **not** protect data once it arrives: not on the device, not in your logs, not in your database. And by default it does not authenticate the *client* (that is mutual TLS, Lesson 6.2).

MOBILE (REACT NATIVE)

Mobile apps sometimes add **certificate pinning** (accepting only specific certificates or keys) to resist interception on hostile networks. It raises the bar for attackers but complicates certificate rotation; a pinning mistake can lock every user out of your app. Use it deliberately, with backup pins.

FAILURE MODES

Using `Math.random()` for secrets. Rolling your own crypto or token format. Reusing an AES-GCM nonce with the same key (catastrophic). Treating base64 as protection. Comparing MACs with `==`. Hard-coding keys in repositories. Disabling TLS verification "temporarily" (`rejectUnauthorized: false`) and shipping it.

Takeaways

- Pick primitives by the guarantee you need: integrity, authenticity, confidentiality, or public verifiability.
- Asymmetric signatures let many parties verify without being able to forge, which is the foundation of federated identity.
- TLS protects the pipe, not the endpoints.

Assessment

SCENARIO 1 · TEN MICROSERVICES

ShopSphere's auth service issues HS256 JWTs and shares the secret with ten microservices so they can verify them. One microservice, maintained by a contractor, is compromised. What can the attacker now do? Propose a better design.

SCENARIO 2 · 'IT'S ENCRYPTED'

A developer stores `{ userId, role }` in a base64-encoded cookie and calls it encrypted. Explain the two separate problems and what the minimum correct fix is.

SCENARIO 3 · PIN OR NOT?

A fintech app's security team wants certificate pinning in the React Native app. List the threat it addresses, the operational risk it creates, and how you would mitigate that risk.

LESSON 1.5

Threat Modeling

Thinking like an attacker before an attacker does it for you.

Learning objectives

- Run a lightweight threat model on any feature using data-flow diagrams and STRIDE.
- Identify assets, entry points and trust boundaries.
- Prioritize risks instead of treating every theoretical issue as urgent.

The problem

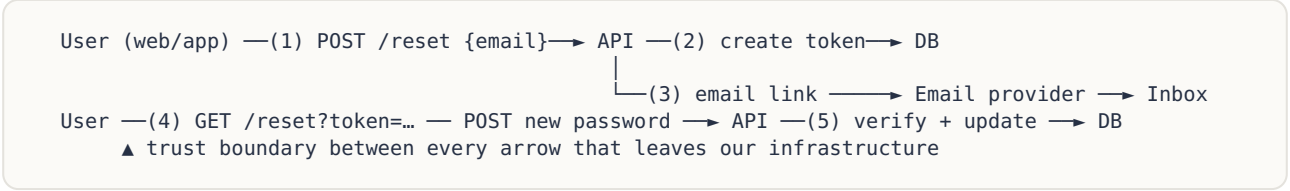
Checklists only cover attacks someone already thought of. Real systems have unique flows, and the vulnerabilities live in the gaps between components. Threat modeling is a structured way to ask four questions (as framed by Adam Shostack, and used widely in industry): *What are we building? What can go wrong? What are we going to do about it? Did we do a good job?*

STRIDE

Originally developed at Microsoft, STRIDE maps neatly onto auth concepts:

Threat	Violates	Auth example
Spoofing	Authentication	Stolen session, forged token, phished password
Tampering	Integrity	Modified JWT claims, changed price in request
Repudiation	Auditability	"I never issued that refund," and no log proves otherwise
Information disclosure	Confidentiality	IDOR leaks orders; verbose errors leak user existence
Denial of service	Availability	Login endpoint flooded with expensive hash checks
Elevation of privilege	Authorization	Support agent performs manager-only refund

Worked example: ShopSphere password reset



Walking STRIDE across it reveals: enumeration (different responses for known/unknown emails), a guessable or non-expiring token, the token leaking via the `Referer` header to analytics scripts on the reset page, the reset link pointing to an attacker's domain if the server builds it from a client-controlled `Host` header (host header poisoning), sessions not invalidated after reset, no rate limiting (email bombing), and no audit trail. Seven real issues from one small diagram. Lesson 2.8 solves them.

Prioritizing

Rank by **likelihood × impact**, and favor fixes that eliminate classes of bugs (a central authorization layer) over point patches. An exploit requiring a stolen hardware key ranks below one requiring only a free account.

INDUSTRY PRACTICE

Mature companies run threat models during design review for any feature touching identity, payments or personal data. Microsoft's Security Development Lifecycle popularized it; tools range from a whiteboard to the free OWASP Threat Dragon and Microsoft Threat Modeling Tool. At startups it is often a 30-minute conversation around a diagram, which is still enormously valuable.

Takeaways

- Draw the data flow, mark trust boundaries, then walk STRIDE across every arrow.
- Every arrow crossing a boundary is an attack surface.
- Prioritize; not every finding is critical.

Assessment

SCENARIO 1 · THREAT-MODEL 'SIGN IN WITH PHONE NUMBER'

ShopSphere's mobile app adds login via SMS one-time code. Draw the flow and identify at least one threat in each STRIDE category.

SCENARIO 2 · PRIORITIZE

Your review found: (a) login errors reveal whether an email exists; (b) admin endpoints lack role checks; (c) session cookies lack `SameSite`; (d) no audit log for refunds. Rank them and justify the order.

PHASE 2

Authentication Mechanics: Web and Mobile

How login actually works: verifying passwords safely, remembering users with sessions and tokens, protecting cookies, storing credentials on browsers and phones, and handling the full account lifecycle.

IN THIS PHASE

1. Password Login Done Right
2. Server-Side Sessions
3. Cookies in Depth and CSRF
4. Tokens and JWTs
5. Sessions vs Tokens: The Real Trade-off
6. Token Storage in the Browser
7. Authentication on Mobile
8. The Account Lifecycle

LESSON 2.1

Password Login Done Right

A login endpoint is the most attacked URL you will ever own.

Learning objectives

- Build a login flow that resists enumeration, brute force and credential stuffing.
- Understand rate limiting at multiple layers and the lockout trade-off.
- Design registration and login responses that leak nothing.

The problem

Attackers rarely "hack" passwords one at a time. They run **credential stuffing**: billions of username/password pairs leaked from other sites, replayed automatically against yours through rotating residential proxies. Because people reuse passwords, a small percentage succeeds, and a small percentage of millions is a lot of accounts. Your login endpoint must be correct *and* hostile to automation.

The flow

```
Client                          Node.js API
POST /login {email, pw} ──►   1. Rate limit (IP, account, global)
                              2. Bot / risk signals (optional)
                              3. Look up user by normalized email
                              4. Verify hash (or a dummy hash if no user)
                              5. Same generic error for every failure
                              6. Success: MFA challenge if enabled
                              7. Create session/tokens, regenerate ID
                              8. Audit log (success AND failure)
          ◄──                 200 + credential  |  401 "Invalid email or password"
```

User enumeration

If login says "No account with that email" for one input and "Wrong password" for another, attackers can harvest which emails are registered, then target them. Enumeration leaks through three channels:

- **Messages:** use one generic error.
- **Timing:** if the user does not exist you skip the slow hash and respond in 2 ms instead of 80 ms. Fix: verify against a precomputed dummy hash.
- **Other endpoints:** registration ("email already taken") and password reset. For registration, respond "Check your email to continue" in both cases and send either a welcome link or an "you already have an account" email.

KEY IDEA

Every endpoint that accepts an identifier is a potential oracle. Ask: does the response, its timing, or its side effects differ depending on whether the account exists?

Rate limiting and lockout

Layer	Stops	Watch out for
Per IP	Simple scripts	Useless against proxy botnets; punishes shared IPs (offices, mobile carriers)
Per account	Targeted brute force	Hard lockout lets attackers lock out victims on purpose
Global / anomaly	Stuffing waves	Needs monitoring: spike in failure ratio
Breached-password check	Stuffing success	Check at signup and login, prompt reset
MFA	Almost everything	The strongest stuffing defense by far

Prefer **progressive delays and step-up challenges** over hard lockouts: after several failures, require a CAPTCHA or add delay, rather than locking the account entirely.

Implementation (Express)

```
import argon2 from "argon2";
import rateLimit from "express-rate-limit";

const DUMMY_HASH = await argon2.hash("not-a-real-password-placeholder");
const loginLimiter = rateLimit({ windowMs: 15 * 60_000, limit: 20, standardHeaders:
true });

app.post("/login", loginLimiter, async (req, res) => {
  const email = String(req.body.email ?? "").trim().toLowerCase();
  const password = String(req.body.password ?? "");

  const user = await db.users.findByEmail(email);
  const ok = await argon2.verify(user?.passwordHash ?? DUMMY_HASH, password);

  if (!user || !ok || user.status !== "active") {
    await audit("login_failed", { email, ip: req.ip });
    await failures.record(email);           // per-account counter for step-up
    return res.status(401).json({ error: "Invalid email or password" });
  }

  await audit("login_success", { userId: user.id, ip: req.ip });
  if (user.mfaEnabled) return startMfaChallenge(req, res, user);
  await createSession(req, res, user);      // Lesson 2.2: new session ID
  res.json({ ok: true });
});
```

Note `user.status !== "active"`: suspended accounts get the same generic error, so status is not leaked either. (Whether to tell a *legitimate* suspended user why is a product decision; do it after authentication, not before.)

WEB (REACT)

Use a real `<form>` with `autocomplete="username"` and `"current-password"` so password managers work, and never log form values to analytics or error tools like Sentry.

MOBILE (REACT NATIVE)

Set `textContentType="password"` / `autoComplete="password"` to enable OS autofill. Mobile carriers put thousands of users behind few IPs (carrier-grade NAT), so IP-only rate limits hurt app users more than web users.

INDUSTRY PRACTICE

Large providers use risk signals beyond the password: device, location, velocity, IP reputation. Managed platforms (Auth0, Okta, Cognito, Clerk) ship breached-password detection and bot protection; this is one of the strongest reasons teams buy rather than build.

Takeaways

- Equalize messages, timing and side effects for existing and non-existing accounts.
- Layer rate limits; prefer step-up over hard lockout.
- MFA and breached-password checks are the real defenses against stuffing.

Assessment

SCENARIO 1 · THE LOCKOUT WEAPON

ShopSphere locks accounts for 24 hours after 5 failed logins. A competitor scripts failures against your top 1,000 sellers on Black Friday. Redesign the policy.

SCENARIO 2 · FIND THE ORACLE

The forgot-password endpoint returns the same message for every email, but takes 600 ms for registered users (it sends an email synchronously) and 20 ms otherwise. Explain the leak and fix it.

SCENARIO 3 · STUFFING IN PROGRESS

At 3 a.m., successful logins are normal but failed logins jumped 40×, spread across 30,000 IPs. What is happening, what do you do in the next hour, and what do you build next week?

LESSON 2.2

Server-Side Sessions

The oldest, simplest and still often the best way to remember a user.

Learning objectives

- Implement a session system: creation, lookup, expiry, logout.
- Prevent session fixation and understand idle vs absolute timeouts.
- Support "see your devices" and "log out everywhere."

How it works

```
LOGIN    API creates random ID → Redis: sess:K7x… = {userId:88, createdAt, lastSeen, ua}
         Set-Cookie: __Host-sid=K7x…; HttpOnly; Secure; SameSite=Lax; Path=/
REQUEST  Cookie: __Host-sid=K7x… → Redis GET sess:K7x… → found & fresh? → req.user = 88
LOGOUT   Redis DEL sess:K7x… ; clear cookie  (instantly effective everywhere)
```

The session ID is a **random, opaque reference**. It contains no data, so stealing the database of session IDs is the only way to learn anything; and it can be revoked instantly by deleting the record.

Design decisions

- **ID entropy:** at least 128 bits from a CSPRNG (`crypto.randomBytes(32)`).
- **Store a hash of the ID** in Redis, not the raw ID, so a Redis dump does not hand out live sessions.
- **Idle timeout** (e.g. 30 min for banking, days for shopping) and **absolute timeout** (re-login after, say, 30 days regardless of activity).
- **Session fixation:** if an attacker can plant a known session ID in a victim's browser *before* login and you keep the same ID after login, the attacker is now logged in as the victim. Always issue a **new ID at login** and at privilege changes.
- **Per-user index** (`user_sessions:88` → set of session hashes) enables "your devices" and "log out everywhere."

Implementation (minimal, educational)

```
import crypto from "node:crypto";
const sha256 = (s) => crypto.createHash("sha256").update(s).digest("hex");
const IDLE = 60 * 60 * 24 * 7;   // 7 days sliding, in seconds

export async function createSession(req, res, user) {
  const old = req.cookies["__Host-sid"];
  if (old) await redis.del(`sess:${sha256(old)}`);           // anti-fixation
  const id = crypto.randomBytes(32).toString("base64url");
  const key = sha256(id);
  const data = { userId: user.id, createdAt: Date.now(), ua: req.get("user-agent") };
  await redis.set(`sess:${key}`, JSON.stringify(data), { EX: IDLE });
  await redis.sAdd(`user_sessions:${user.id}`, key);
  res.cookie("__Host-sid", id, { httpOnly: true, secure: true, sameSite: "lax", path: "/",
maxAge: IDLE * 1000 });
}

export async function requireSession(req, res, next) {
  const id = req.cookies["__Host-sid"];
  const raw = id && (await redis.get(`sess:${sha256(id)}`));
  if (!raw) return res.status(401).json({ error: "unauthenticated" });
  const s = JSON.parse(raw);
  if (Date.now() - s.createdAt > 30 * 86400_000) return res.status(401).end(); // absolute
  await redis.expire(`sess:${sha256(id)}`, IDLE);           // slide idle timeout
  req.user = await loadUser(s.userId);                        // fresh status/roles
  if (!req.user || req.user.status !== "active") return res.status(401).end();
  next();
}
```

In production, use a maintained library such as `express-session` with a Redis store, or your framework's built-in sessions, but you should understand everything they do, which is the code above.

KEY IDEA

Sessions trade a database lookup per request for instant revocation and always-fresh user data. For most monoliths, that is an excellent trade.

MOBILE (REACT NATIVE)

Sessions work for mobile apps too: send the opaque ID as a bearer value in the `Authorization` header and store it in secure storage. "Session" describes *server-side state*, not "cookies."

Failure modes

Predictable IDs; no regeneration at login; logout that only deletes the cookie client-side (the session stays valid server-side); sessions surviving a password change; unbounded session lifetime; Redis without authentication exposed to the network.

Takeaways

- Opaque random IDs referencing server-side state; regenerate at login; idle plus absolute timeouts.

- Logout means deleting server state, not just a cookie.

Assessment

SCENARIO 1 · SCALING SESSIONS

ShopSphere grows from one API server to twelve behind a load balancer and users randomly get logged out. Explain why, compare sticky sessions with a shared store, and choose.

SCENARIO 2 · PASSWORD CHANGED

Implement, in words, "changing your password logs out every other device but keeps the current one." Which data structures do you need?

SCENARIO 3 · REDIS OUTAGE

Redis goes down. Every request now fails authentication. Should you fail open or closed? What design reduces the blast radius?

LESSON 2.3

Cookies in Depth and CSRF

The browser's automatic credential is a gift and a liability.

Learning objectives

- Configure every security-relevant cookie attribute and explain each.
- Understand "site" vs "origin" and how SameSite works.
- Explain CSRF from first principles and choose layered defenses.

Cookie attributes

Attribute	Effect	Recommendation
`HttpOnly`	JavaScript cannot read it	Always, for session cookies
`Secure`	Sent only over HTTPS	Always
`SameSite=Lax`	Not sent on cross-site subrequests; sent on top-level GET navigations	Good default
`SameSite=Strict`	Never sent cross-site, even when following a link	Sensitive apps; breaks "arrive logged in" from email links
`SameSite=None`	Always sent (requires `Secure`)	Only for genuine cross-site needs (embedded widgets)
`Domain`	Omit it = host-only. Setting it shares with all subdomains	Omit unless you must share
`Path`	Limits by path; not a security boundary	`/`
`Max-Age` / `Expires`	Persistence; omit for a browser-session cookie	Match your session policy
`__Host-` prefix	Browser enforces Secure, Path=/, no Domain	Use for session cookies

Origin vs site. An *origin* is scheme + host + port (`https://shop.com:443`). A *site* is roughly the registrable domain (`shop.com`), so `api.shop.com` and `www.shop.com` are **same-site but cross-origin**. SameSite cares about *site*; CORS cares about *origin*. Confusing them causes many bugs.

CSRF from first principles

```
1. Victim logs in to shop.com          → browser stores sid cookie
2. Victim visits evil.com (still logged in)
3. evil.com contains:
     <form action="https://shop.com/account/email" method="POST">
       <input name="email" value="attacker@evil.com"></form>
     <script>document.forms[0].submit()</script>
4. Browser sends POST to shop.com AND attaches the sid cookie automatically
5. shop.com sees a valid session → changes email → attacker resets password
```

The attacker never sees the cookie. They just make the victim's browser use it. **CSRF exists because the browser attaches cookies based on the destination, not on who initiated the request.**

Defenses, in layers

1. **SameSite=Lax/Strict** blocks most cross-site POSTs. Strong baseline, but not complete: same-site subdomains (a compromised `blog.shop.com`) are not "cross-site," and Lax allows top-level GETs, so **never change state on GET**.
2. **Check `Origin` / `Sec-Fetch-Site`** on state-changing requests. Modern browsers send `Sec-Fetch-Site: same-origin | same-site | cross-site | none`.
3. **CSRF tokens** (synchronizer token or signed double-submit) for defense in depth, especially for legacy browsers or same-site subdomain risks.
4. **Require JSON with `Content-Type: application/json`**: plain HTML forms cannot send it cross-site without a CORS preflight. Useful, but not a sole defense.

```
// Reject cross-site state-changing requests
function csrfGuard(req, res, next) {
  if (["GET", "HEAD", "OPTIONS"].includes(req.method)) return next();
  const site = req.get("sec-fetch-site");
  const origin = req.get("origin");
  const allowed = site ? ["same-origin", "same-site"].includes(site)   // tighten to same-
origin if possible
                       : origin === "https://shop.com";
  if (!allowed) return res.status(403).json({ error: "cross-site request blocked" });
  next();
}
```

CORS IS NOT CSRF PROTECTION

CORS controls whether a page on another origin may *read the response*. A cross-site form POST is sent regardless, and the damage is done before any response is read. Also never reflect arbitrary `Origin` values with `Access-Control-Allow-Credentials: true`; that lets any site read your users' data.

MOBILE (REACT NATIVE)

Native apps calling your API with an `Authorization` header are not subject to CSRF. But an in-app WebView that shares cookies with the system browser can be. Be deliberate about which surfaces use cookies.

Takeaways

- `__Host-` prefix, HttpOnly, Secure, SameSite=Lax as the default session cookie.
- CSRF = the browser spends your credential for someone else. Layer SameSite, origin checks and tokens. No state changes on GET.

Assessment

SCENARIO 1 · THE SUBDOMAIN

ShopSphere's marketing team runs a WordPress blog at `blog.shop.com`, which gets compromised. Session cookies use `SameSite=Lax` and `Domain=shop.com`. What can the attacker do, and which two changes help most?

SCENARIO 2 · LOGOUT VIA GET

`GET /logout` logs the user out. Why is this a CSRF issue even though it's "harmless," and what about `GET /cart/add?item=…` ?

SCENARIO 3 · THE CORS FIX

A developer fixes a CORS error by setting `Access-Control-Allow-Origin` to the request's `Origin` header with credentials allowed. Explain the vulnerability with a concrete attack.

LESSON 2.4

Tokens and JWTs

Self-contained, signed claims: powerful, popular and frequently misused.

Learning objectives

- Read a JWT and explain each part and claim.
- Validate a JWT completely and in the right order.
- Recognize the classic JWT attacks and why they work.

Anatomy

A JWT (RFC 7519) is three base64url segments: `header.payload.signature`.

```
eyJhbGciOiJFZERTQSIsImtpZCI6ImsyMDI2In0 . eyJzdWIiOiI4OCIsImF1ZCI6InNob3AtYXBpIi... .
3nF9...
{ "alg": "EdDSA", "kid": "k2026" }        { "iss": "https://auth.shop.com",
                                            "sub": "88", "aud": "shop-api",
                                            "iat": 1793500000, "exp": 1793500900,
                                            "scope": "orders:read", "jti": "a1b2" }
```

Claim	Meaning	Validate?
`iss`	Who issued it	Must equal your expected issuer
`sub`	Who it is about	Use as the principal ID
`aud`	Who it is for	Must include *your* service
`exp` / `nbf`	Validity window	Yes, with small clock-skew tolerance
`iat`	Issued at	Useful for "issued before password change" checks
`jti`	Unique ID	Needed for denylists / replay detection

KEY IDEA

A JWT is signed, not encrypted. Anyone holding it can read the payload (paste one into jwt.io). Never put secrets or sensitive personal data in it.

Complete validation

1. Parse the header; **reject any `alg` not on your allowlist** (never trust the token to choose its algorithm).
2. Select the key by `kid` from *your* trusted key set (never from a URL in the token, such as `jku` / `x5u`).
3. Verify the signature.

4. Check `exp`, `nbf` (with ~30–60 s skew), `iss`, `aud`.
5. Check token type if you issue several kinds (access vs ID token) so one cannot be used as another.
6. *Then* authorize based on claims, and consider whether claims may be stale.

```
import { jwtVerify, createRemoteJWKSet, SignJWT } from "jose";

const JWKS = createRemoteJWKSet(new URL("https://auth.shop.com/.well-known/jwks.json"));

export async function verifyAccessToken(token) {
  const { payload } = await jwtVerify(token, JWKS, {
    issuer: "https://auth.shop.com",
    audience: "shop-api",
    algorithms: ["EdDSA"],      // explicit allowlist
    clockTolerance: 30,
  });
  return payload;               // { sub, scope, ... }
}

// Issuing (auth service only)
export const issue = (userId, privateKey) =>
  new SignJWT({ scope: "orders:read" })
    .setProtectedHeader({ alg: "EdDSA", kid: "k2026", typ: "at+jwt" })
    .setIssuer("https://auth.shop.com").setAudience("shop-api")
    .setSubject(String(userId)).setIssuedAt().setExpirationTime("15m")
    .sign(privateKey);
```

The classic attacks

Attack	How	Prevention
`alg: none`	Attacker strips the signature, sets alg none; naive library accepts	Algorithm allowlist
Algorithm confusion	Server expects RS256; attacker sends HS256 signed with the *public key* as the HMAC secret	Allowlist; key bound to algorithm
`kid` injection	`kid` used in a file path or SQL query	Treat `kid` as a lookup key in a fixed map
`jku` / `x5u`	Token points to attacker's key URL	Ignore; use configured JWKS only
`decode` instead of `verify`	Code reads claims without checking signature	Code review rule: never `decode` for auth
Missing `aud` check	Token for service A accepted by service B	Always check audience
Weak HS256 secret	Offline brute force of the secret	256-bit random secrets, or use asymmetric

THE MOST COMMON REAL BUG

`const user = jwt.decode(token)` in middleware. It "works" in every test, because tests use valid tokens. It accepts any forged token in production.

Takeaways

- JWTs carry signed, readable claims; validate algorithm, signature, issuer, audience and time, in that spirit.
- Follow RFC 8725 (JWT Best Current Practices). Prefer asymmetric algorithms (EdDSA, ES256, RS256) across services.

Assessment

SCENARIO 1 · CODE REVIEW

Review: `const p = jwt.verify(token, process.env.PUBLIC_KEY)` with no options, in a service that accepts RS256 tokens. Explain the risk precisely and fix it.

SCENARIO 2 · ROLE IN THE TOKEN

Access tokens carry `role: "seller"` and live 24 hours. A seller is banned for fraud. What happens, for how long, and what are three ways to shrink that window?

SCENARIO 3 · TWO SERVICES

The `orders` and `payments` services both accept tokens from the same issuer. A token issued for `orders` works on `payments`. Why, and what should change in both issuing and validation?

LESSON 2.5

Sessions vs Tokens: The Real Trade-off

It is not "old vs modern." It is revocation vs locality.

Learning objectives

- Compare stateful sessions and stateless tokens on the dimensions that actually matter.
- Design access/refresh token systems with rotation and reuse detection.
- Choose a revocation strategy consciously.

The real trade-off

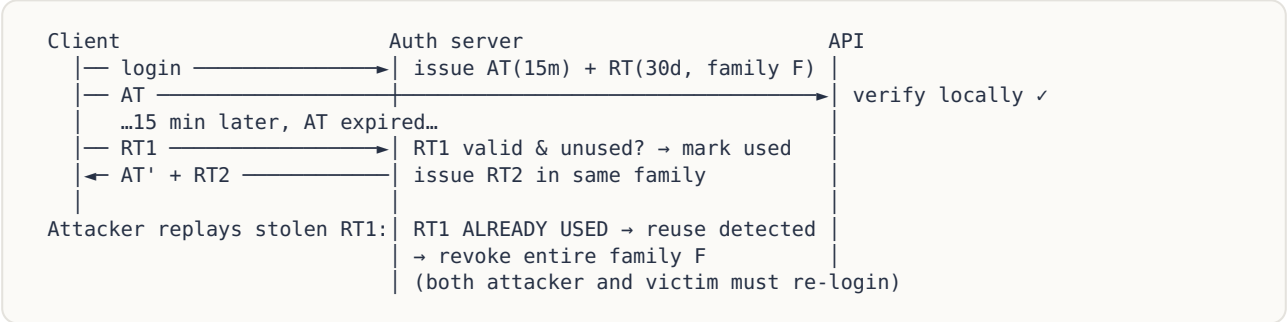
Dimension	Server sessions	Stateless access tokens
Revocation	Instant	Only at expiry, unless you add state back
Per-request cost	Store lookup	Signature check, no network
Data freshness	Always current	Claims frozen at issue time
Cross-service use	Every service needs the store	Any service with the public key
Third-party APIs	Awkward	Natural (OAuth)
Complexity	Low	Higher (rotation, refresh, keys)

KEY IDEA

Every "stateless" system that needs instant logout ends up adding state back (denylists, token versions, session-backed refresh). The question is how much state, and where.

The access + refresh pattern

Short-lived **access tokens** (5–15 minutes) are presented to APIs and validated locally. A long-lived **refresh token**, sent *only* to the auth server, obtains new access tokens. The refresh token is the thing you revoke, so revocation takes effect within one access-token lifetime.



Rotation with reuse detection turns a stolen refresh token into a detectable event: legitimate and malicious clients cannot both keep using it.

```
async function refresh(presentedToken) {
  const rec = await db.refreshTokens.findByHash(sha256(presentedToken));
  if (!rec) throw unauthenticated();
  if (rec.usedAt) {                                  // replay!
    await db.refreshTokens.revokeFamily(rec.familyId);
    await audit("refresh_reuse_detected", { userId: rec.userId });
    throw unauthenticated();
  }
  if (rec.expiresAt < Date.now() || rec.revoked) throw unauthenticated();
  await db.refreshTokens.markUsed(rec.id);           // must be atomic (conditional
update)
  const next = crypto.randomBytes(32).toString("base64url");
  await db.refreshTokens.insert({ hash: sha256(next), familyId: rec.familyId, userId:
rec.userId,
                                  expiresAt: Date.now() + 30 * 86400_000 });
  return { accessToken: await issue(rec.userId, privateKey), refreshToken: next };
}
```

Real systems add a short **grace period** for races (two tabs refreshing simultaneously), and make `markUsed` a conditional update so two concurrent refreshes cannot both succeed.

Revocation strategies

- **Short access-token TTL**: simplest; accept minutes of staleness.
- **Token version per user**: store `tokenVersion` on the user; embed it; bump to invalidate all tokens (requires a cheap lookup, often cached).
- `jti` **denylist** in Redis until expiry for specific tokens.
- **Opaque access tokens + introspection** (RFC 7662): fully stateful, centrally revocable.

INDUSTRY PRACTICE

The most common production architecture is a hybrid: a session or refresh token held server-side as the source of truth, plus short-lived JWT access tokens for APIs and microservices. Identity providers like Auth0, Okta and Microsoft Entra ID document refresh token rotation and reuse detection as standard features.

Takeaways

- Choose by revocation needs, data freshness and service topology, not fashion.
- Short access tokens + rotating, revocable refresh tokens is the standard compromise.

Assessment

SCENARIO 1 · CHOOSE

A single Node.js monolith serves a React site only, 50k users, small team. A teammate wants JWTs "because they're modern." Make the case for your choice.

SCENARIO 2 · THE TWO-TAB RACE

Users with two tabs open are randomly logged out after rotation was added. Explain exactly why and propose two fixes with their security cost.

SCENARIO 3 · INSTANT BAN

Legal requires that banned users lose access within 10 seconds across 30 microservices validating JWTs locally. Design it.

LESSON 2.6

Token Storage in the Browser

Where you keep the keycard decides what an XSS bug can steal.

Learning objectives

- Compare localStorage, memory and HttpOnly cookies against XSS and CSRF.
- Understand what XSS can do *even when* tokens are unreachable.
- Design a Backend-for-Frontend (BFF).

The options

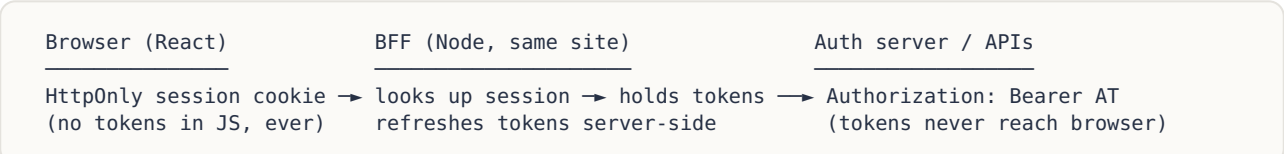
Storage	XSS can steal it?	Survives refresh?	CSRF exposure
`localStorage` / `sessionStorage`	**Yes**, trivially	Yes / per tab	No
JS memory (variable)	Harder, still possible while page runs	No (needs silent refresh)	No
HttpOnly cookie	**No**	Yes	Yes, mitigate (2.3)

KEY IDEA

HttpOnly does not make XSS harmless. Injected script cannot *steal* the cookie, but it can still *use* it by making requests from the victim's page while it runs. What HttpOnly prevents is theft: exfiltrated tokens that work from the attacker's machine for days.

So: prevent XSS first (React's escaping, no `dangerouslySetInnerHTML` with untrusted input, a strict **Content Security Policy**, dependency hygiene), and *then* minimize what XSS can take.

The Backend-for-Frontend pattern



The browser only holds an HttpOnly, SameSite session cookie. The BFF, a server you control, holds OAuth access and refresh tokens and attaches them to upstream calls. The IETF's *OAuth 2.0 for Browser-Based Applications* guidance describes BFF as the most secure architecture for browser apps; frameworks like Next.js make it natural because server code already sits next to the React app.

```
// React: no token handling at all
const res = await fetch("/api/orders", { credentials: "include" });

// BFF route (Express): attach the server-held token upstream
app.get("/api/orders", requireSession, csrfGuard, async (req, res) => {
  const at = await tokens.getFreshAccessToken(req.session);   // refreshes if needed
  const upstream = await fetch("https://api.shop.com/orders", {
    headers: { Authorization: `Bearer ${at}` },
  });
  res.status(upstream.status).send(await upstream.text());
});
```

FAILURE MODES

Refresh tokens in localStorage (one XSS = long-term account takeover). Third-party scripts (analytics, chat widgets) running on pages that hold tokens. `dangerouslySetInnerHTML` with user content. CSP absent or full of `unsafe-inline`.

Takeaways

- Keep tokens out of JavaScript where possible; BFF + HttpOnly cookies is the strongest browser pattern.
- XSS prevention is still job one; storage choice limits blast radius.

Assessment

SCENARIO 1 · DEFEND THE CHOICE

A blog post says "store JWTs in localStorage; cookies are vulnerable to CSRF." Write the reply you would give in a design review.

SCENARIO 2 · PURE SPA

ShopSphere's React app is a static SPA on a CDN with no server. You can't add a BFF this quarter. What is the least-bad token strategy, and which compensating controls do you add?

LESSON 2.7

Authentication on Mobile

Phones have better secure storage than browsers, and different threats.

Learning objectives

- Store tokens correctly in React Native using Keychain/Keystore.
- Understand what biometrics actually prove.
- Know what app attestation does and does not do.
- Implement a robust token refresh flow on mobile.

The mobile environment

Mobile apps are **public clients**: they cannot keep a secret from a determined user. But they have advantages over browsers: no cross-site requests, no third-party scripts on your screens, and hardware-backed secure storage.

Storage	Encrypted?	Use for tokens?
`AsyncStorage`	No (plain files/SQLite)	**Never**
iOS Keychain	Yes, hardware-backed (Secure Enclave keys)	Yes
Android Keystore-backed encrypted storage	Yes, can be hardware-backed	Yes
`expo-secure-store` / `react-native-keychain`	Wrap the above	Yes

Implementation

```
import * as SecureStore from "expo-secure-store";
import axios from "axios";

export const tokenStore = {
  save: (rt) => SecureStore.setItemAsync("refresh_token", rt,
          { keychainAccessible: SecureStore.WHEN_UNLOCKED_THIS_DEVICE_ONLY }),
  load: () => SecureStore.getItemAsync("refresh_token"),
  clear: () => SecureStore.deleteItemAsync("refresh_token"),
};

let accessToken = null;           // memory only
let refreshing = null;            // single-flight promise

const api = axios.create({ baseURL: "https://api.shop.com" });
api.interceptors.request.use((cfg) => {
  if (accessToken) cfg.headers.Authorization = `Bearer ${accessToken}`;
  return cfg;
});
api.interceptors.response.use(undefined, async (err) => {
  const original = err.config;
  if (err.response?.status !== 401 || original._retried) throw err;
  original._retried = true;
  refreshing ??= (async () => {                // many 401s → ONE refresh call
    const rt = await tokenStore.load();
    const { data } = await axios.post("https://auth.shop.com/token/refresh", {
refreshToken: rt });
    accessToken = data.accessToken;
    await tokenStore.save(data.refreshToken);   // rotation
  })().finally(() => { refreshing = null; });
  try { await refreshing; } catch { await logoutLocally(); throw err; }
  return api(original);
});
```

The single-flight promise matters doubly with rotation: five parallel 401s triggering five refreshes would look like token reuse and revoke the whole family (Lesson 2.5).

What biometrics prove

Face ID or fingerprint is verified **locally by the OS**. Your server never sees it. Using `expo-local-authentication` to show a prompt and then reading a token proves only that *someone passed a prompt in your UI*, which is bypassable on a rooted device.

The stronger pattern: store the token (or a private key) in Keychain/Keystore **with an access control that requires biometric authentication to release it**, so the OS, not your JavaScript, enforces it. The strongest pattern is passkeys (Lesson 3.2), where biometrics unlock a private key that signs a server challenge.

App attestation

Apple App Attest and **Google Play Integrity** let your backend check that a request comes from a genuine, unmodified install on a real device. This fights bots and modified clients. It is **not user authentication**, it can fail for legitimate users, and it should feed a risk decision rather than act as an absolute gate.

OTHER MOBILE SPECIFICS

Use the system browser (ASWebAuthenticationSession / Custom Tabs) for OAuth, not an embedded WebView (Lesson 4.3). Prefer **universal links / app links** over custom URL schemes for redirects, since any app can claim a custom scheme. Clear tokens on logout *and* revoke them server-side. Remember that Keychain items can survive app uninstall on iOS; handle first-launch cleanup.

Takeaways

- Tokens go in Keychain/Keystore, never AsyncStorage; access token in memory.
- Biometrics are local; bind them to key release, or use passkeys.
- Attestation authenticates the app instance, not the user.

Assessment

SCENARIO 1 · THE BIOMETRIC TOGGLE

The app has "Unlock with Face ID": on launch, it calls `authenticateAsync()` and, if successful, reads the refresh token from AsyncStorage. List every flaw and redesign it.

SCENARIO 2 · STOLEN PHONE

A user's unlocked phone is stolen. What can the thief do in your app, and what server-side features let the user limit damage from the web?

SCENARIO 3 · BOT FARM

Fake accounts are being created through your mobile signup API by scripts that mimic the app. The PM asks for a "secret app key." Propose a better approach.

LESSON 2.8

The Account Lifecycle

Login is one moment. Verification, recovery, changes and deletion are where accounts are actually stolen.

Learning objectives

- Design secure email verification, password reset and email change flows.
- Implement session invalidation on security events.
- Treat account recovery as an authentication path with equal rigor.

Password reset done right

Solving the threat model from Lesson 1.5:

```
POST /forgot {email} → always "If an account exists, we've sent a link" (async email, constant time)
  token = randomBytes(32)  → store sha256(token), userId, expires = now+30min, used=false
  link  = https://shop.com/reset#token=…   ← built from CONFIG base URL, never the Host header
POST /reset {token, newPassword}
  find by sha256(token) → unexpired & unused? → set password (hashed), mark used,
  revoke ALL sessions + refresh tokens, notify user by email, audit log
```

Details that matter: put the token in the URL *fragment* (`#`) or exchange it immediately so it does not leak via `Referer` and logs; keep third-party scripts off the reset page; single use; short expiry; rate-limit by account and IP.

```
async function createReset(email) {
  const user = await db.users.findByEmail(email);
  if (!user) return;                                  // same response either way
  const token = crypto.randomBytes(32).toString("base64url");
  await db.resets.insert({ userId: user.id, hash: sha256(token), expiresAt: Date.now() +
30 * 60_000 });
  await queue.sendEmail(user.email, `${config.APP_URL}/reset#token=${token}`); // async
}
```

Other lifecycle flows

- **Email verification:** same token pattern. Do not grant trust (e.g. merging accounts, receiving refunds) to unverified emails.
- **Changing email:** require recent re-authentication, confirm via the *new* address, and notify the *old* address with a way to revert. Attackers who hijack a session usually change the email first.
- **Changing password:** require the current password (or recent re-auth); revoke other sessions.
- **Log out everywhere:** delete all sessions and refresh-token families; bump token version.

- **Account recovery** (lost MFA device): this is an authentication path and often the weakest one. Use recovery codes generated at MFA setup, cooldown periods, and notifications. Support agents who "verify" callers over the phone are a classic social-engineering target.
- **Deletion:** revoke everything, handle data retention obligations (GDPR, Lesson 7.4).

FAILURE MODE

Recovery flows routinely bypass strong authentication. An account protected by a passkey but recoverable via SMS is only as strong as SMS.

Takeaways

- Every lifecycle flow is an authentication flow. Apply the same rigor as login.
- Security events (password reset/change, MFA change, email change) must revoke sessions and notify the user.

Assessment

SCENARIO 1 · HOST HEADER

The reset email link is built with `` `https://${req.headers.host}/reset?token=…` ``. Describe the attack end to end.

SCENARIO 2 · SUPPORT DESK

A caller says they lost their phone with the authenticator app and asks support to remove MFA. Design a recovery policy that balances security, cost and user frustration.

SCENARIO 3 · SESSION HIJACK, THEN WHAT?

An attacker has a victim's session cookie for 10 minutes. List the actions they will try in order to make access permanent, and which controls block each.

PHASE 3

Stronger Authentication

Passwords alone fail at internet scale. This phase covers multi-factor authentication, phishing-resistant passkeys, passwordless flows, and risk-based authentication that adapts to context.

IN THIS PHASE

1. Multi-Factor Authentication
2. Passkeys, WebAuthn and FIDO2
3. Passwordless Flows
4. Adaptive and Risk-Based Authentication

LESSON 3.1

Multi-Factor Authentication

Combining independent evidence, and understanding which factors survive phishing.

Learning objectives

- Explain why MFA works and what "independent factors" means.
- Compare SMS, TOTP, push and hardware factors on real attack resistance.
- Implement TOTP and understand its weaknesses.

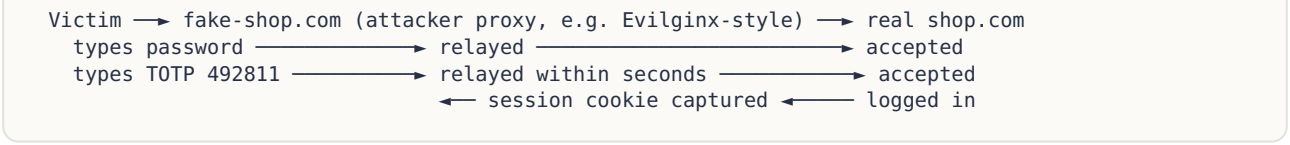
Why it works

An attacker with a leaked password still needs a second, independent piece of evidence. Microsoft has publicly reported that MFA blocks the overwhelming majority (over 99%) of automated account-compromise attacks. But not all MFA is equal, because attackers adapted.

The factor ladder

Factor	Stops stuffing	Stops real-time phishing	Main weakness
SMS / voice OTP	Yes	**No**	SIM swap, SS7 interception, phishing relay
Email OTP	Yes	No	Email account compromise
TOTP app (Google Authenticator, Authy)	Yes	**No**	Phishing proxy relays the code in real time
Push approval	Yes	Partly	**MFA fatigue**: spamming prompts until the user taps "Approve"
Push with number matching	Yes	Better	Still phishable via a real-time proxy
Passkey / security key (FIDO2)	Yes	**Yes**	Recovery flows; device loss

How phishing defeats OTPs



The code is valid; it is simply typed into the wrong site. Only factors that **cryptographically bind to the website's origin** (FIDO2/passkeys) defeat this. In 2022, Uber publicly confirmed a breach in which an attacker reportedly obtained a contractor's password and then spammed MFA push requests until one was accepted, the textbook MFA-fatigue attack.

TOTP under the hood

Both sides share a secret (from the QR code). Each 30-second window, both compute `HMAC-SHA1(secret, floor(time/30))` and truncate it to 6 digits (RFC 6238, built on HOTP RFC 4226).

```
// Educational implementation. Use a maintained library in production.
import crypto from "node:crypto";
function totp(secret /* Buffer */, t = Date.now(), step = 30) {
  const counter = Buffer.alloc(8);
  counter.writeBigUInt64BE(BigInt(Math.floor(t / 1000 / step)));
  const h = crypto.createHmac("sha1", secret).update(counter).digest();
  const off = h[h.length - 1] & 0x0f;
  const code = (h.readUInt32BE(off) & 0x7fffffff) % 1_000_000;
  return String(code).padStart(6, "0");
}
// Verify current window ±1 for clock drift; reject reuse of the same code (store last used step).
```

Server-side requirements: encrypt the TOTP secret at rest (it must be readable, so it cannot be hashed); rate-limit attempts (a 6-digit code has only a million possibilities); prevent replay within the window; generate **recovery codes** at enrollment (hash them like passwords).

KEY IDEA

MFA turns "steal one secret" into "steal two independent things," but only phishing-resistant MFA stops an attacker who controls the website the user is typing into.

WEB (REACT)

Use `autocomplete="one-time-code"` on OTP inputs. Treat the period between password success and MFA success as a *pending* state, never a logged-in state: a half-authenticated session must not access any API.

MOBILE (REACT NATIVE)

`textContentType="oneTimeCode"` (iOS) and SMS Retriever / autofill (Android) enable OTP autofill. If your own app is the push-approval authenticator, add number matching and show location/device context in the prompt.

Takeaways

- Any MFA beats none; phishing-resistant MFA beats the rest.
- SMS is the weakest acceptable option; NIST 800-63B-4 treats it as restricted.
- Recovery and enrollment are part of the MFA system.

Assessment

SCENARIO 1 · THE PENDING STATE

After password success, ShopSphere issues a full session cookie and redirects to the MFA page. The MFA page then marks the session "verified." What can an attacker with the password do?

SCENARIO 2 · CHOOSE FACTORS

Choose MFA options for (a) ShopSphere customers in India, many on low-end Android phones, and (b) ShopSphere staff with refund powers. Justify each.

SCENARIO 3 · BRUTE-FORCING TOTP

The MFA endpoint has no rate limit. Estimate how practical brute-forcing a 6-digit code is, and design the limits.

LESSON 3.2

Passkeys, WebAuthn and FIDO2

The first mainstream credential that cannot be phished, reused or leaked from your database.

Learning objectives

- Explain public-key authentication and why it resists phishing.
- Walk through WebAuthn registration and authentication ceremonies.
- Understand synced vs device-bound passkeys and their trade-offs.

The idea

Instead of a shared secret, the user's device generates a **key pair per website**. The server stores only the **public key**. To log in, the server sends a random **challenge**; the device signs it with the private key after a local user check (biometric or PIN). Nothing reusable crosses the network, and your database holds nothing an attacker can log in with.

The phishing resistance comes from the browser: it includes the **origin** in the signed data and only offers credentials scoped to the site's **Relying Party ID** (e.g. `shop.com`). On `fake-shop.com` the passkey simply is not available.

The ceremonies

```
REGISTRATION
Server ── options {challenge, rp:{id:"shop.com"}, user:{id}, pubKeyCredParams} ──► Browser/App
Browser → authenticator: create key pair for shop.com (user verifies with Face ID/PIN)
Browser ── {credentialId, publicKey, attestation, clientDataJSON(origin, challenge)} ──► Server
Server: check challenge, origin, rpId hash → store {credentialId, publicKey, signCount, userId}

AUTHENTICATION
Server ── {challenge, rpId, allowCredentials?} ──► Browser/App
Authenticator signs (challenge + origin + rpId hash) with private key after user verification
Browser ── {credentialId, signature, authenticatorData, clientDataJSON} ──► Server
Server: look up publicKey by credentialId → verify signature, challenge, origin, flags → session
```

Implementation sketch (SimpleWebAuthn)

```
// server
import { generateAuthenticationOptions, verifyAuthenticationResponse } from
"@simplewebauthn/server";

app.post("/passkey/options", async (req, res) => {
  const options = await generateAuthenticationOptions({ rpID: "shop.com",
userVerification: "required" });
  await challenges.save(req, options.challenge);      // bind to this login attempt, short
TTL
  res.json(options);
});

app.post("/passkey/verify", async (req, res) => {
  const cred = await db.passkeys.findById(req.body.id);
  const { verified, authenticationInfo } = await verifyAuthenticationResponse({
    response: req.body,
    expectedChallenge: await challenges.take(req),     // single use
    expectedOrigin: "https://shop.com",
    expectedRPID: "shop.com",
    credential: { id: cred.id, publicKey: cred.publicKey, counter: cred.counter },
  });
  if (!verified) return res.status(401).end();
  await db.passkeys.updateCounter(cred.id, authenticationInfo.newCounter);
  await createSession(req, res, await loadUser(cred.userId));
  res.json({ ok: true });
});
// browser: import { startAuthentication } from "@simplewebauthn/browser";
// const resp = await startAuthentication({ optionsJSON }); then POST resp to /passkey/
verify
```

Library APIs change between major versions; check current docs. The *checks* (challenge, origin, RP ID, signature) do not change.

Synced vs device-bound

	Synced passkeys (iCloud Keychain, Google Password Manager, 1Password)	Device-bound (YubiKey, some enterprise policies)
Survive phone loss	Yes	No, need a backup key
Phishing resistant	Yes	Yes
Security ceiling	Tied to cloud account security	Highest
Typical use	Consumers	Admins, high-assurance workforce

MOBILE (REACT NATIVE)

Passkeys on iOS and Android require **associated domains** (`apple-app-site-association`) and **Digital Asset Links** (`assetlinks.json`) so the OS knows your app may use credentials for `shop.com` . Community libraries such as `react-native-passkey` bridge the native APIs. Web and app then share the same passkeys.

INDUSTRY PRACTICE

Google, Apple and Microsoft support passkeys across their platforms, and many consumer services (Amazon, PayPal, GitHub, and others) let users sign in with them. Typical rollouts add passkeys alongside passwords, prompt users after a successful login to "create a passkey," and only later reduce password reliance. Recovery remains the hard part.

Takeaways

- Passkeys replace shared secrets with per-site key pairs; servers store public keys only.
- Origin binding by the browser/OS is what defeats phishing.
- Your passkey security is capped by your weakest recovery path.

Assessment

SCENARIO 1 · EXPLAIN TO THE CEO

In five sentences a non-engineer would follow, explain why passkeys stop the phishing attack that defeated your SMS codes.

SCENARIO 2 · DOMAIN MIGRATION

ShopSphere plans to move from `shopsphere.in` to `shopsphere.com` . What happens to existing passkeys, and how do you plan the migration?

SCENARIO 3 · CHALLENGE REUSE

A developer stores the WebAuthn challenge in a global variable instead of per login attempt. What breaks, and what attack could it enable?

LESSON 3.3

Passwordless Flows

Magic links and one-time codes: easier for users, and full of subtle traps.

Learning objectives

- Design magic-link and OTP login securely.
- Recognize cross-device, email-scanner and session-binding problems.
- Know when passwordless is the right product choice.

How it works

The user enters an email or phone number; the server sends a one-time link or code; possession of the inbox or phone is the proof. Security is now exactly as strong as the user's email account or SIM.

The traps

- **Email security scanners** in corporate mailboxes "click" every link to scan it, consuming single-use tokens. Fix: link opens a page with a "Confirm sign-in" button (POST), or use a code instead.
- **Cross-device:** user requests on laptop, opens email on phone; who gets logged in? Decide deliberately. Safer default: the login completes on the device that *requested* it (the phone shows "approved, return to your laptop"), with the requesting session polling.
- **Login CSRF / session binding:** an attacker can send *their own* magic link to a victim, logging the victim into the attacker's account (who then enters payment details). Bind the link to the browser that requested it (a cookie set at request time).
- **OTP brute force:** a 6-digit code needs strict attempt limits per code and per identifier.
- **SMS pumping fraud:** bots trigger OTPs to premium-rate numbers you pay for. Rate-limit by phone prefix and use bot defenses.

```
Browser A: POST /magic {email} → server sets cookie pending=P1, emails link with token T (bound to P1)
User clicks link → GET /magic/confirm?t=T → page with [Confirm] button → POST
Server: T valid & unused & cookie pending == P1 ? → session on this browser
        cookie missing/different → "Approve sign-in on the device that requested it" flow
```

MOBILE (REACT NATIVE)

Use universal links / app links so the magic link opens the app, and verify the binding the same way. Phone OTP is often the right primary method for mobile-first markets, with the risks above managed explicitly.

Takeaways

- Passwordless moves trust to the email or phone provider. That is often a fine trade, but it is a trade.

- Bind links to the requesting session, make them single-use and short-lived, and defend against scanners and pumping.

Assessment

SCENARIO 1 · THE DISAPPEARING LINKS

Enterprise customers report magic links are "already used" when they click them. Diagnose and fix.

SCENARIO 2 · LOGIN CSRF

Explain how an attacker benefits from logging a victim into the attacker's ShopSphere account, and which design element prevents it.

LESSON 3.4

Adaptive and Risk-Based Authentication

Matching friction to risk, request by request.

Learning objectives

- Use context signals to adjust authentication requirements.
- Design step-up authentication for sensitive actions.
- Balance fraud prevention, privacy and false positives.

The idea

Rather than one fixed bar, evaluate risk per login and per action. Signals include: known vs new device, impossible travel, IP reputation (data centers, Tor, known proxies), velocity (many accounts from one device), behavioral anomalies, and the sensitivity of the action. Low risk: no friction. Medium: step-up. High: block and notify.

Step-up authentication

```
Session (auth_time = 2 hours ago, method = password)
User: POST /payouts/bank-account (sensitive)
API policy: requires method ∈ {passkey, totp} AND auth_time < 5 min
  → 401 { error: "step_up_required", acr: "mfa", max_age: 300 }
Client shows MFA prompt → re-auth → session updated (auth_time = now, method = passkey)
Retry → allowed
```

Track `auth_time` and the **authentication method** in the session or token (OpenID Connect standardizes `auth_time`, `acr` and `amr` claims). Sensitive actions check them. This is the "re-enter your password to change your email" pattern, generalized.

INDUSTRY PRACTICE

Large consumer platforms publicly describe risk-based challenges, such as Google's sign-in checks on unfamiliar devices, and banks commonly require extra verification for new payees. Managed platforms (Okta, Auth0, Microsoft Entra Conditional Access, AWS Cognito advanced security) sell adaptive authentication because building good risk signals needs data most companies do not have.

FAILURE MODES

Risk engines that fail open silently. Signals that discriminate against users on mobile carriers, VPNs or shared devices. Step-up checks implemented in the UI but not the API. Device "remember me" cookies that never expire and are not bound to the account.

Takeaways

- Authentication strength is per action, not per session.
- Record how and when the user authenticated; let policies require recency and method.

Assessment

SCENARIO 1 · DESIGN THE POLICY

Define risk tiers and responses for ShopSphere: browsing, placing an order, changing the delivery address, adding a payout bank account (sellers), and issuing a refund (staff).

SCENARIO 2 · FALSE POSITIVES

After launching impossible-travel detection, support tickets triple, mostly from users on mobile networks. What is likely happening, and how do you tune it?

PHASE 4

Delegation and Federated Identity

How one system lets another act on a user's behalf, and how users sign in with an identity owned by someone else. OAuth 2.0, PKCE, OpenID Connect, SAML, and the attacks against them.

IN THIS PHASE

1. The Delegation Problem
2. OAuth 2.0 Roles and Flows
3. PKCE and Public Clients
4. OpenID Connect
5. Scopes, Audiences and Consent
6. SAML and Enterprise SSO
7. OAuth Attacks and OAuth 2.1

LESSON 4.1

The Delegation Problem

Why the industry invented OAuth instead of just asking for passwords.

Learning objectives

- Explain the password anti-pattern OAuth replaced.
- Separate *delegated authorization* from *authentication*.
- Describe what a well-scoped, revocable delegation looks like.

The problem

Before OAuth, if a photo-printing site wanted your Google Photos, it asked for your **Google password**. That gave it total, permanent access: email, documents, the ability to change your password. You could not limit it, see it, or revoke it without changing your password everywhere. If the printing site was breached, your entire Google account was too.

What users needed was a **valet key**: a key that drives the car but cannot open the trunk, that you hand over without giving away your house keys, and that you can cancel at any time.

> **KEY IDEA**
>
> **OAuth 2.0 (RFC 6749) is a framework for delegated authorization: a user grants an application limited access to their resources on another service, without sharing their password. On its own it is not an authentication protocol.**

What good delegation has

- **No password sharing**: the user authenticates only with the service that owns the data.
- **Limited scope**: "read photos," not "everything."
- **Limited lifetime**: short-lived access tokens.
- **Revocable**: the user can revoke the app from their account settings.
- **Auditable**: the service knows which app did what.

Real-world scenarios

- A calendar app reading your Google Calendar.
- A CI service (e.g. a deployment tool) accessing your GitHub repositories with specific permissions.
- ShopSphere's sellers connecting their accounting software to ShopSphere to sync orders: ShopSphere becomes the *provider* of an OAuth API (Lesson 4.5).

Why "OAuth for login" is subtle

If an app uses "I received an access token for your Google account" as proof that you logged in, it is misusing an authorization artifact as an authentication one. An access token says "the bearer may call this API," not "this user is present at *this* app." Lesson 4.4 shows how OpenID Connect fixes this.

Takeaways

- OAuth replaces password sharing with scoped, revocable, short-lived delegated access.
- Delegated authorization is not authentication.

Assessment

SCENARIO 1 · THE INTEGRATION

An analytics startup asks ShopSphere sellers for their ShopSphere email and password to "import sales data." List the risks to sellers and to ShopSphere, and outline what ShopSphere should offer instead.

SCENARIO 2 · ACCESS TOKEN AS LOGIN

A developer says, "If Google gives us an access token, the user is obviously authenticated." Explain what is wrong with this reasoning.

LESSON 4.2

OAuth 2.0 Roles and Flows

Four roles, a handful of grant types, and why most of them are now discouraged.

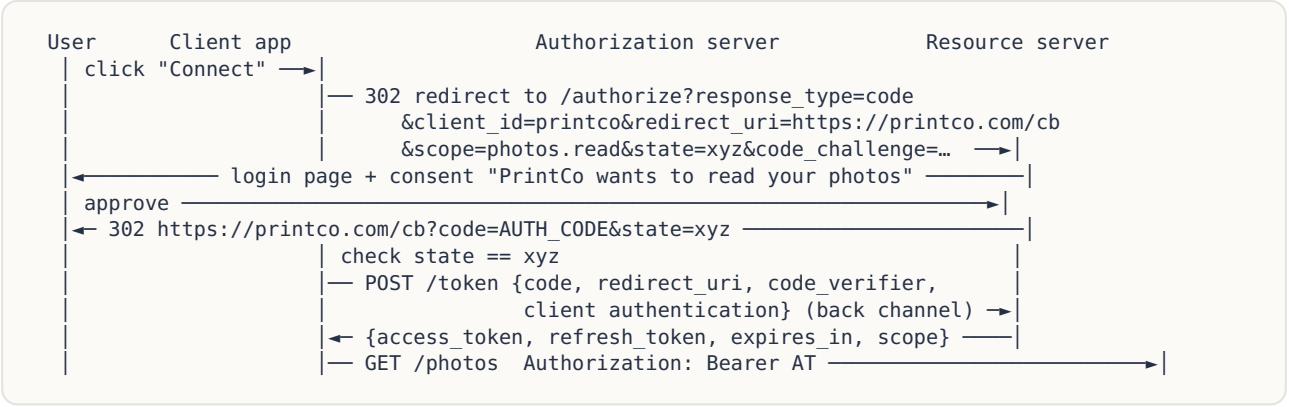
Learning objectives

- Name the four OAuth roles and map them onto real systems.
- Walk through the authorization code flow step by step.
- Know which grants to use today and which are deprecated.

The four roles

Role	Meaning	Example
Resource owner	The user who owns the data	Priya
Client	The app wanting access	Photo-printing site, ShopSphere mobile app
Authorization server	Authenticates the user, issues tokens	accounts.google.com, Auth0, Keycloak
Resource server	The API holding the data	Google Photos API, ShopSphere API

The authorization code flow



Why two steps (code, then token)? The redirect travels through the browser (**front channel**), where URLs leak into history, logs and other apps. The code is short-lived, single-use and useless without the client's credentials or PKCE verifier. The tokens travel over a direct server-to-server request (**back channel**).

Grant types today

Grant	Use	Status
Authorization code + PKCE	Any app acting for a user: web, SPA, mobile	**Recommended**

Grant	Use	Status
Client credentials	Machine-to-machine, no user	Recommended
Device authorization (RFC 8628)	TVs, CLIs: "go to example.com/device and enter ABCD-1234"	Recommended for input-limited devices
Refresh token	Renew access	Recommended, with rotation for public clients
Implicit	Tokens directly in the redirect URL	**Deprecated**: tokens leak via URLs
Resource owner password	App collects the user's password	**Deprecated**: defeats OAuth's purpose

Confidential vs public clients. A server-side app can keep a client secret (confidential). SPAs and mobile apps cannot (public): anything shipped to users is readable. Public clients use PKCE instead of a secret (Lesson 4.3).

INDUSTRY PRACTICE

Every major identity platform (Google, Microsoft Entra ID, Okta/Auth0, AWS Cognito, Keycloak) implements authorization code with PKCE as the default for user-facing apps. Most teams use a certified library (openid-client for Node, AppAuth for native) rather than hand-coding the protocol.

Takeaways

- Authorization code + PKCE for users; client credentials for machines; device flow for input-limited devices.
- Front channel carries only a short-lived code; tokens come over the back channel.
- Implicit and password grants are deprecated.

Assessment

SCENARIO 1 · MAP THE ROLES

A seller connects accounting software "LedgerLy" to their ShopSphere account so LedgerLy can read orders nightly. Identify each OAuth role and choose the grant type.

SCENARIO 2 · NIGHTLY JOB

LedgerLy's nightly sync runs with no user present. Which tokens does it need, how long should they live, and what happens when the seller revokes access?

SCENARIO 3 · THE SMART TV

ShopSphere launches a smart TV app for browsing deals. Design its login.

LESSON 4.3

PKCE and Public Clients

How apps that cannot keep secrets can still use OAuth safely.

Learning objectives

- Explain the authorization code interception attack.
- Implement PKCE and understand why it works.
- Apply correct OAuth practices for SPAs and React Native apps.

The attack PKCE solves

On mobile, the redirect back to the app can be intercepted by a malicious app registered for the same custom URL scheme (`shopsphere://callback`). In browsers, codes can leak through logs or referrers. Without a client secret, whoever obtains the code can redeem it.

How PKCE works (RFC 7636)

```
App: code_verifier  = random 43–128 chars (kept in memory)
     code_challenge = BASE64URL(SHA256(code_verifier))
/authorize?…&code_challenge=…&code_challenge_method=S256   ──► server stores challenge with the code
/token { code, code_verifier }                              ──► server checks SHA256(verifier) == challenge
A thief with only the code cannot produce the verifier → redemption fails.
```

```
// Works in modern browsers and React Native (with a crypto polyfill/library)
function base64url(bytes) {
  return btoa(String.fromCharCode(...bytes)).replace(/\+/g, "-").replace(/\//g,
"_").replace(/=+$/, "");
}
export async function createPkce() {
  const verifier = base64url(crypto.getRandomValues(new Uint8Array(32)));
  const digest = await crypto.subtle.digest("SHA-256", new TextEncoder().encode(verifier));
  return { verifier, challenge: base64url(new Uint8Array(digest)) };
}
```

RFC 9700 recommends PKCE for **all** clients, including confidential ones, because it also protects against code injection attacks.

MOBILE (REACT NATIVE)

Use the **system browser** for login (`ASWebAuthenticationSession` on iOS, Custom Tabs on Android), never an embedded WebView: the user can see the real URL, already-signed-in sessions work, and your app cannot read the password. RFC 8252 (*OAuth 2.0 for Native Apps*) codifies this. Libraries: `react-native-app-auth` (wraps AppAuth), or `expo-auth-session` / `expo-web-browser` in Expo apps. Prefer universal/app links for the redirect URI. Store the refresh token in Keychain/Keystore.

WEB (REACT)

A pure SPA can run authorization code + PKCE in the browser, but then tokens live in JavaScript (Lesson 2.6). The more secure choice is a BFF that acts as a confidential client and keeps tokens server-side, giving the browser only an HttpOnly session cookie.

Takeaways

- PKCE binds the code to the app instance that started the flow.
- Native apps: system browser + PKCE + secure storage. SPAs: prefer a BFF.

Assessment

SCENARIO 1 · THE WEBVIEW

A teammate wants "Sign in with Google" inside a React Native WebView for a smoother UX. Give three reasons against it, including one that is not about security.

SCENARIO 2 · `PLAIN` METHOD

A library defaults to `code_challenge_method=plain`. What protection is lost and in which scenarios does it matter?

LESSON 4.4

OpenID Connect

The identity layer on top of OAuth: how "Sign in with Google" actually proves who you are.

Learning objectives

- Explain what OIDC adds to OAuth 2.0.
- Validate ID tokens correctly, including `nonce`.
- Implement social login and understand account-linking risks.

What OIDC adds

OpenID Connect adds `scope=openid`, an **ID token** (a JWT stating who authenticated, when, how, and *for which client*), a standard **UserInfo** endpoint, and **discovery** (`/.well-known/openid-configuration`) listing endpoints and keys.

	ID token	Access token
Audience	The **client** app	The **API** (resource server)
Purpose	"This user authenticated, for you"	"The bearer may call this API"
Send to APIs?	**No**	Yes

Validating an ID token

Verify signature (keys from the provider's JWKS), `iss` equals the provider, `aud` contains **your client_id**, `exp` / `iat`, and `nonce` equals the value you generated for this login (prevents replay/injection). Then identify the user by `iss` + `sub`, never by email alone.

```
import * as client from "openid-client";   // v6 API; check docs for your version
const config = await client.discovery(new URL("https://accounts.google.com"), CLIENT_ID,
CLIENT_SECRET);
// After redirect back:
const tokens = await client.authorizationCodeGrant(config, currentUrl, {
  pkceCodeVerifier: verifier, expectedState: state, expectedNonce: nonce,
});
const claims = tokens.claims();             // validated ID token claims
const user = await accounts.findOrCreateByFederatedId(claims.iss, claims.sub, claims);
```

ACCOUNT LINKING PITFALL

"Find the existing account with the same email and log them in" enables takeover if the identity provider does not verify emails, or lets users set arbitrary ones. Only auto-link on `email_verified: true` from a provider you trust for that domain; otherwise require the user to log into the existing account first and link explicitly.

MOBILE (REACT NATIVE)

"Sign in with Apple" is required by App Store rules in many cases when an app offers other third-party social logins; check current App Review Guidelines. Native SDKs (Google Sign-In, Apple) return an ID token; send it to your backend, verify it there, and issue *your own* session or tokens. Never trust identity claims the app passes without server verification.

INDUSTRY PRACTICE

Google, Microsoft, Apple, Okta and others are OpenID providers. Enterprise "Sign in with Microsoft" for B2B SaaS is OIDC against Microsoft Entra ID. Use certified OIDC libraries; the OpenID Foundation runs a certification program.

Takeaways

- OIDC = OAuth + ID token + discovery + UserInfo. ID tokens are for clients; access tokens are for APIs.
- Users are identified by `iss` + `sub`. Link accounts carefully.

Assessment

SCENARIO 1 · ID TOKEN TO THE API

The React app sends the Google ID token to ShopSphere's API on every request, and the API checks the signature. List what is wrong and propose the correct architecture.

SCENARIO 2 · EMAIL-BASED LINKING

ShopSphere adds "Sign in with GitHub" and links accounts by email. Describe a takeover scenario and a safe linking design.

LESSON 4.5

Scopes, Audiences and Consent

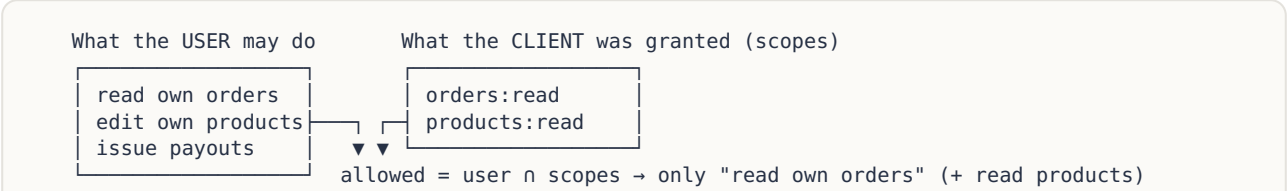
Designing what tokens may do, for whom, and with whose permission.

Learning objectives

- Design scopes for an API you provide.
- Use audience to stop token misuse across services.
- Understand that scopes limit the *client*, not the *user*.

Scopes limit the app, not the person

A scope like `orders:read` says what the **client** may do on the user's behalf. Your API must still check what the **user** may do. Effective permission is the **intersection**:



Designing scopes for ShopSphere's partner API

- Resource-based and verb-based: `orders:read`, `products:write`.
- Coarse enough to understand on a consent screen, fine enough to follow least privilege.
- Never `admin` or `*` for third parties.
- Plan for **incremental authorization**: request more scopes only when needed.

Audience

Include `aud` in tokens and have each API reject tokens not meant for it. Without it, a token issued for a low-trust service could be replayed against a high-trust one. **Resource indicators** (RFC 8707) let clients request tokens for a specific API.

Consent

Consent screens should be honest and specific. Users should be able to view and revoke connected apps. First-party apps (your own React and RN apps) usually skip consent; third-party apps must not.

CONSENT PHISHING

Attackers register an OAuth app with a convincing name and trick users into granting it mailbox or file access, bypassing passwords and MFA entirely. Providers counter with publisher verification and admin consent policies (e.g. Microsoft Entra ID lets administrators restrict which apps users can approve).

Takeaways

- Effective permission = user's rights ∩ client's scopes. Check both.
- Always set and validate audience.

Assessment

SCENARIO 1 · SCOPE DESIGN

Design the scope list for ShopSphere's partner API covering orders, products, inventory, payouts and customer data. Which scopes would you never grant to third parties?

SCENARIO 2 · SCOPE-ONLY CHECK

The orders API checks only `scope.includes("orders:read")` and returns the order by ID. What is the vulnerability?

LESSON 4.6

SAML and Enterprise SSO

The XML-based protocol that still runs much of enterprise single sign-on.

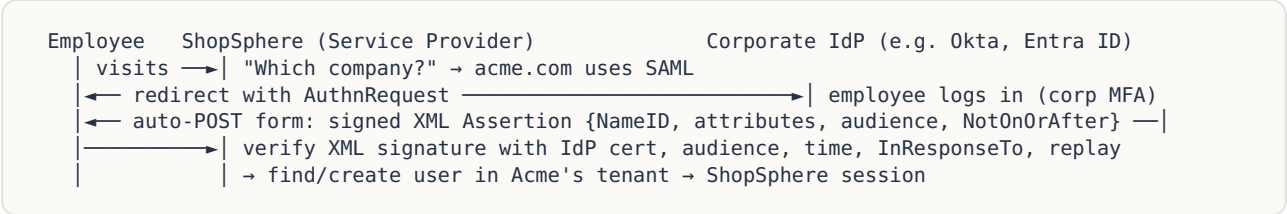
Learning objectives

- Explain SAML's roles and the SP-initiated flow.
- Understand why enterprises require SSO from SaaS vendors.
- Recognize SAML-specific vulnerabilities.

Why it matters

When ShopSphere sells to large businesses, their IT departments will ask: "Do you support SSO with our identity provider (Okta, Microsoft Entra ID, Google Workspace)?" They want employees to use corporate accounts, central MFA policies, and **instant offboarding**: disable one account and access to every app ends. Without SSO, enterprise deals stall.

SAML in one diagram



	SAML 2.0	OpenID Connect
Format	XML assertions	JSON / JWT
Era	2005, enterprise	2014, web and mobile
Mobile friendliness	Poor	Good
Where you'll meet it	Enterprise SSO, legacy apps	Everything new; also enterprise

SAML-specific risks

XML signature wrapping (attacker moves the signed element and injects an unsigned one that the application reads), XML parser vulnerabilities (XXE), accepting unsigned assertions, missing audience/time checks, and replay. Parsing and verifying SAML correctly is hard; use well-maintained libraries, or a managed product. Several widely used SAML libraries across languages have had serious signature-bypass vulnerabilities over the years.

INDUSTRY PRACTICE

"SSO tax" is a public debate: many SaaS vendors put SAML SSO only in expensive enterprise tiers. B2B startups often adopt products like WorkOS, Auth0/Okta, Microsoft Entra External ID, or open-source Keycloak/BoxyHQ rather than implementing SAML themselves, because each enterprise IdP has quirks.

Takeaways

- Enterprises need SSO for central control and offboarding; it is a revenue requirement for B2B.
- SAML is XML-signature-based and easy to get subtly wrong; buy or use proven libraries.

Assessment

SCENARIO 1 · OFFBOARDING

An Acme employee is fired and disabled in Okta at 10:00. They still have a ShopSphere session valid until 18:00. What can you do about this, and what does SCIM (Lesson 7.1) add?

SCENARIO 2 · BUILD OR BUY

Three enterprise prospects require SAML within six weeks. You have two backend engineers. Argue for an approach.

LESSON 4.7

OAuth Attacks and OAuth 2.1

The real-world ways OAuth deployments break, and the modern consolidated rules.

Learning objectives

- Recognize redirect URI, state, mix-up and token leakage attacks.
- Apply the RFC 9700 security best current practice.
- Understand what OAuth 2.1 changes.

The attack catalogue

Attack	Mechanism	Defense
Open redirect / loose redirect URI	Server allows `https://shop.com/*` or prefix matching; attacker uses an open redirector to leak the code	**Exact string matching** of pre-registered redirect URIs
CSRF on callback	Attacker completes flow with *their* code in the victim's browser (account linking to attacker)	`state` bound to the user's session, or PKCE
Code injection	Stolen code redeemed in attacker's session	PKCE
Mix-up	Client using multiple IdPs is tricked into sending a code to the wrong one	`iss` parameter in responses (RFC 9207), per-IdP redirect URIs
Token leakage via URL	Implicit flow, tokens in logs/history/Referer	Code flow only
Bearer token replay	Stolen token used elsewhere	Short TTL; sender-constrained tokens: **DPoP** (RFC 9449) or mTLS
Refresh token theft (public clients)	Long-lived token stolen from device	Rotation + reuse detection, or sender-constraining

OAuth 2.1

OAuth 2.1 is an IETF effort that consolidates OAuth 2.0 with a decade of security lessons. Its main points: PKCE required for authorization code; implicit and password grants removed; exact redirect URI matching; no bearer tokens in query strings; refresh tokens for public clients must be sender-constrained or rotated. Implementing RFC 9700 today gets you essentially there.

KEY IDEA

Most OAuth breaches are not cryptographic failures. They are configuration failures: loose redirect URIs, missing state, tokens in URLs, overly broad scopes.

Testing OAuth

Try modified `redirect_uri` values (subdomains, paths, `@`, encoded characters); remove or reuse `state`; replay codes; swap codes between sessions; request extra scopes; check where tokens end up (logs, browser history, analytics).

Takeaways

- Exact redirect URIs, PKCE everywhere, state/nonce, no tokens in URLs, short-lived and ideally sender-constrained tokens.

Assessment

SCENARIO 1 · WILDCARD REDIRECT

A partner asks you to allow `https://*.partner.com/callback` as a redirect URI because they have many subdomains. Explain the risk and offer alternatives.

SCENARIO 2 · REVIEW A FLOW

A web app's callback handler: reads `code` from the URL, exchanges it for tokens, stores the tokens in localStorage, and redirects to `?next=` from the query string. Find at least four problems.

PHASE 5

Authorization Models

Authentication tells you who; authorization decides what. This phase covers where checks belong, the classic models (RBAC, ABAC, ReBAC), policy engines, multi-tenant SaaS, and what frontends can and cannot do.

IN THIS PHASE

1. Authorization Fundamentals
2. Role-Based Access Control
3. Attribute-Based Access Control and Policy Engines
4. Relationship-Based Access Control
5. Multi-Tenant Authorization
6. Frontend vs Backend Authorization

LESSON 5.1

Authorization Fundamentals

Deny by default, check every object, and centralize the decision.

Learning objectives

- Distinguish function-level and object-level authorization.
- Apply deny-by-default and least privilege.
- Structure code so authorization cannot be forgotten.

Two levels of checks

- **Function-level:** may this principal call this *kind* of operation at all? ("Only managers can refund.") Missing it is OWASP API Security's *Broken Function Level Authorization*.
- **Object-level:** may this principal act on *this specific object*? ("This refund belongs to an order in your region.") Missing it is **BOLA**, ranked first in the OWASP API Security Top 10.

Add a third that is easy to miss: **property-level**. A seller may update their product's `price` but not its `commissionRate`. Blindly assigning request bodies to models (**mass assignment**) breaks this.

Where authorization lives

```
Request ─► Authentication middleware (who?)   → req.user
        ─► Route-level check (may call this?) → e.g. requireRole("manager")
        ─► Load resource scoped to principal  → WHERE id = ? AND seller_id = ?
        ─► Policy check on the object         → can(user, "update", product)
        ─► Allowlisted fields only            → pick(body, ["title","price"])
        ─► Audit decision                      → who, what, allowed/denied
```

Implementation patterns

```
// A single policy function, used everywhere. Deny by default.
const policies = {
  product: {
    read:   (u, p) => p.published || p.sellerId === u.id || u.roles.includes("support"),
    update: (u, p) => p.sellerId === u.id,
    delete: (u, p) => p.sellerId === u.id || u.roles.includes("admin"),
  },
};
export function can(user, action, type, resource) {
  const rule = policies[type]?.[action];
  return rule ? Boolean(rule(user, resource)) : false;      // unknown = deny
}

app.patch("/products/:id", requireUser, async (req, res) => {
  const product = await db.products.findById(req.params.id);
  if (!product || !can(req.user, "update", "product", product))
    return res.status(404).end();                           // don't confirm existence
  const changes = pick(req.body, ["title", "description", "price"]); // no mass assignment
  res.json(await db.products.update(product.id, changes));
});
```

Better still, **scope queries by the principal** so a missing check returns nothing rather than someone else's data: `db.orders.findOne({ id, customerId: req.user.id })`.

> **KEY IDEA**
>
> **Authorization that depends on every developer remembering an `if` in every handler will eventually be forgotten. Make the secure path the default path: centralized policies, scoped queries, deny-by-default routing, and tests that prove it.**

Testing

Automated **authorization matrix tests**: for every endpoint × role × ownership combination, assert the expected outcome. Tools like Burp Suite's Autorize extension replay requests with a lower-privileged user's session to find gaps.

Takeaways

- Check function, object and property levels. Deny by default.
- Centralize policy, scope queries, allowlist writable fields, test the matrix.

Assessment

SCENARIO 1 · MASS ASSIGNMENT

`PATCH /users/me` does `db.users.update(req.user.id, req.body)`. What can a user do? Fix it two different ways.

SCENARIO 2 · MAKE IT UNFORGETTABLE

ShopSphere has 140 endpoints and three IDOR bugs found last quarter. Propose structural changes so the fourth never ships.

LESSON 5.2

Role-Based Access Control

The workhorse model: simple, auditable, and prone to role explosion.

Learning objectives

- Model permissions with roles, and separate roles from permissions.
- Recognize role explosion and its remedies.
- Apply separation of duties.

Model

Users get **roles**; roles bundle **permissions**; code checks **permissions**, not roles.

```
users ──< user_roles >── roles ──< role_permissions >── permissions
Priya ── support ──────────────── orders:read, customers:read
Arjun ── manager ──────────────── orders:read, orders:refund, customers:read
Code:  requirePermission("orders:refund")   ← not  requireRole("manager")
```

Checking permissions means adding a "senior support" role later requires no code changes.

Limits

RBAC cannot easily say "managers can refund **orders in their own region** up to **₹50,000**." Teams try `manager_mumbai_under_50k` roles and get **role explosion**: hundreds of roles nobody understands. That is the signal to add attributes (5.3) or relationships (5.4). Note also that "seller" is not really a role for *editing products*; ownership is a relationship.

Separation of duties: the person who creates a vendor should not be the one who approves paying it. RBAC can enforce mutually exclusive roles.

> **INDUSTRY PRACTICE**
>
> Cloud platforms expose RBAC directly: Azure RBAC, Google Cloud IAM roles, Kubernetes RBAC. GitHub organizations use roles (owner, member) plus repository-level permissions. Almost every B2B SaaS ships "Admin / Member / Viewer" first.

Takeaways

- Roles group permissions; code checks permissions.
- When roles start encoding data (region, amount, owner), switch models.

Assessment

SCENARIO 1 · EXPLOSION

ShopSphere now has 87 roles. Analyze how this happens and propose a migration path that does not break production.

LESSON 5.3

Attribute-Based Access Control and Policy Engines

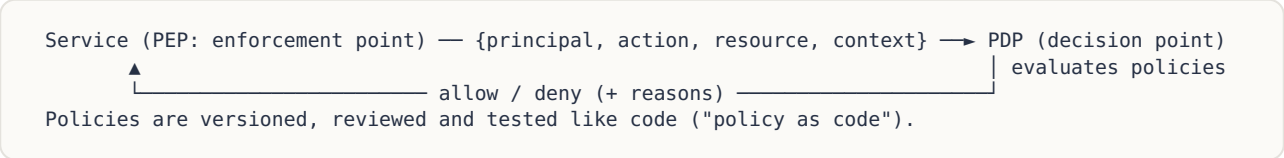
Decisions computed from attributes, expressed as policy rather than scattered code.

Learning objectives

- Express rules over subject, resource, action and environment attributes.
- Understand policy decision/enforcement points.
- Compare OPA/Rego and Cedar.

The model

A decision is a function of attributes: `allow(subject.attrs, action, resource.attrs, environment)`. "Managers may refund orders in their region below ₹50,000 during business hours" is one rule, not hundreds of roles.



Two popular engines

Open Policy Agent (OPA), a CNCF graduated project, uses the Rego language and is widely used for Kubernetes admission, API authorization and infrastructure policy.

```
package shop.refunds
default allow := false
allow if {
  input.action == "refund"
  "manager" in input.principal.roles
  input.resource.region == input.principal.region
  input.resource.amount < 50000
}
```

Cedar, open-sourced by AWS and used by Amazon Verified Permissions, is designed to be readable and formally analyzable.

```
permit (principal, action == Action::"refund", resource)
when {
  principal.role == "manager" &&
  resource.region == principal.region &&
  resource.amount < 50000
};
```

Trade-offs

Flexible and expressive, but: attribute data must be fresh and available at decision time; policies can become hard to reason about ("why was I denied?"); a remote PDP adds latency and a failure point (run it as a sidecar or library, cache carefully, and **fail closed**).

Takeaways

- ABAC handles context-rich rules without role explosion.
- Separate decision from enforcement; treat policies as tested, reviewed code.

Assessment

SCENARIO 1 · WRITE THE POLICY

Write (in plain English or pseudo-policy) rules for: support can view orders but not customer payment details; managers refund within region under ₹50,000; refunds above that need a second approver.

SCENARIO 2 · PDP DOWN

The central policy service is unreachable. Checkout calls it for every request. What should happen, and how should the architecture change?

LESSON 5.4

Relationship-Based Access Control

When permissions follow the graph: owners, teams, folders and sharing.

Learning objectives

- Model permissions as relationships between objects.
- Understand Google Zanzibar's design and its open-source descendants.
- Recognize when ReBAC is worth its complexity.

The problem

"Can Priya view this document?" In Google Docs the answer depends on whether Priya is the owner, an editor, a member of a group that was granted access, or whether the *parent folder* was shared with her organization. Permissions are a **graph**, and roles or attributes model it awkwardly.

Zanzibar

Google's 2019 USENIX ATC paper, *Zanzibar: Google's Consistent, Global Authorization System*, publicly describes the system behind permissions for services such as Drive, YouTube and Calendar. Permissions are stored as **relation tuples**:

```
document:roadmap#owner@user:priya
document:roadmap#parent@folder:q4
folder:q4#viewer@group:eng#member
group:eng#member@user:arjun
```

A schema says "viewers of a document include its editors and the viewers of its parent folder." A check (`can arjun view document:roadmap?`) walks the graph: arjun → member of eng → viewer of folder q4 → parent of roadmap → **allowed**. The paper also describes consistency tokens ("zookies") that prevent the "new enemy" problem: checking permissions against stale data right after access was removed.

```
// OpenFGA-style model
type user
type store
  relations
    define owner: [user]
    define staff: [user]
type product
  relations
    define store: [store]
    define editor: owner from store or staff from store
    define viewer: [user:*] or editor
```

Open-source and managed descendants include **OpenFGA** (a CNCF project originally from Auth0/Okta), **SpiceDB** (AuthZed), and **Ory Keto**.

When to use it

Good fit: sharing, nested hierarchies (orgs → workspaces → projects → items), collaboration products. Overkill: a small app with three roles. Cost: a new critical service, data synchronization between your database and the tuple store, and new mental models for the team.

Takeaways

- ReBAC models ownership and sharing as a graph of relationships.
- Zanzibar demonstrated it at global scale; OpenFGA and SpiceDB make it accessible.

Assessment

SCENARIO 1 · MODEL IT

ShopSphere adds "stores" with multiple staff, and staff can be "store managers" or "catalog editors." Products belong to stores. Write the relationships and the check for "can user X edit product Y?"

SCENARIO 2 · NEW ENEMY

A store owner removes a fired employee, then immediately uploads confidential supplier pricing. The employee can still see it for a few seconds. Explain why and how Zanzibar-style systems address it.

LESSON 5.5

Multi-Tenant Authorization

One platform, thousands of customer organizations, and zero data leaks between them.

Learning objectives

- Design tenant isolation at the identity, application and data layers.
- Handle users who belong to multiple organizations.
- Choose isolation levels and understand their trade-offs.

The problem

B2B SaaS serves many organizations ("tenants") from one system. The worst bug a SaaS company can have is showing Company A's data to Company B. Every request needs **two** authorization answers: is this user in this tenant, and what may they do within it?

Design

```
Token/session: { sub: "u_88", tenant: "org_acme", roles_in_tenant: ["admin"] }
Request: GET /orgs/org_acme/orders/123
  1. Path tenant == token tenant (or user is a member of it)?
  2. Query ALWAYS scoped: WHERE tenant_id = 'org_acme' AND id = 123
  3. Database defense in depth: PostgreSQL Row-Level Security on tenant_id
```

Isolation	How	Pros	Cons
Shared tables + `tenant_id`	Every row tagged	Cheapest, simplest ops	One missed filter = leak; use RLS
Schema per tenant	Separate schemas	Stronger separation	Migrations at scale hurt
Database per tenant	Separate DBs	Strongest, data residency	Expensive, complex

Users in multiple orgs (consultants, agencies): the session should have an *active tenant*, switching tenants should be explicit, and permissions are evaluated per tenant. Enterprise tenants often bring their own IdP (SSO, Lesson 4.6), so login must first resolve "which organization is this?" (by email domain or org-specific URL).

FAILURE MODES

Tenant ID taken from a request header or body without membership check. Caches keyed without tenant (`cache["orders:123"]`). Background jobs and exports that run without tenant context. Search indexes shared across tenants. Support "impersonation" tools without audit.

Takeaways

- Tenant isolation must exist in identity, application queries, caches, jobs and ideally the database itself.

Assessment

SCENARIO 1 · THE CACHE LEAK

After adding Redis caching for order lookups, a customer reports seeing another company's order. Hypothesize the bug and the systemic fix.

SCENARIO 2 · SUPPORT IMPERSONATION

Design a secure "log in as customer" feature for ShopSphere's B2B support team.

LESSON 5.6

Frontend vs Backend Authorization

What React and React Native should do with permissions, and what they must never be trusted with.

Learning objectives

- Use permissions in the UI for experience without relying on it for security.
- Deliver permissions to clients efficiently.
- Handle permission changes during a session.

Two different jobs

The **backend enforces**. The **frontend reflects**: hides buttons a user cannot use, shows "request access" flows, avoids confusing 403s. Both read from the same policy source, but only one is trusted.

```
// API returns what the UI needs to render correctly
// GET /me → { id: "88", permissions: ["orders:read", "products:write"] }

function RefundButton({ order }) {
  const { can } = usePermissions();          // derived from /me
  if (!can("orders:refund")) return null;    // UX only
  return <button onClick={() => refund(order.id)}>Refund</button>;
}
// The /refund endpoint still checks orders:refund server-side, always.
```

For object-level UI decisions, have the API return per-object capabilities: `{ id: 123, ..., _can: { refund: false, edit: true } }`, computed by the same server policy.

Pitfalls

- Shipping admin features in the main web bundle "hidden" behind a flag. The code and endpoints are discoverable. Split bundles or serve admin separately.
- Mobile apps caching permissions for days. If a role is revoked, the UI lies until refresh; the server must still deny. Refresh permissions on app foreground and on 403.
- Hiding sensitive fields in the UI while the API still returns them. If the user may not see it, the API must not send it.

Takeaways

- Frontend authorization is UX; backend authorization is security.
- Compute capabilities on the server and send them down; never send data the user may not see.

Assessment

SCENARIO 1 · HIDDEN FIELDS

The React order page hides the customer's phone number from support agents with CSS. Explain the problem and the correct fix at the API layer.

SCENARIO 2 · STALE PERMISSIONS

A manager is demoted, but the React Native app still shows the Refund button for hours. Is this a security bug? What should happen when they tap it, and how do you improve the experience?

PHASE 6

System Design and Distributed Identity

Identity across many services, regions and clients: gateways, token propagation, service identity, zero trust, key management, revocation at scale, and complete system designs for web and mobile products.

IN THIS PHASE

1. Auth in Monoliths vs Microservices
2. Service-to-Service Identity
3. Zero Trust
4. Designing an Auth Service
5. Revocation, Caching and Consistency at Scale
6. System Design Challenges

LESSON 6.1

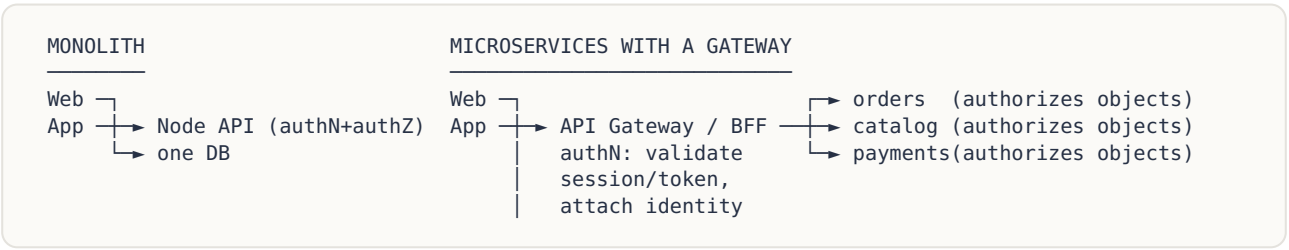
Auth in Monoliths vs Microservices

Where authentication happens, where authorization happens, and how identity travels between services.

Learning objectives

- Compare edge, per-service and hybrid authentication.
- Propagate identity between services safely.
- Avoid the "trusted internal network" trap.

The architectures



Authenticate at the edge, authorize in the service. The gateway validates the external credential once (session cookie, OAuth token) and forwards a trustworthy identity. Each service makes its own object-level decisions because only it knows its data ("does this order belong to this user?").

Propagating identity

Approach	How	Risk
Plain headers (`X-User-Id: 88`)	Gateway sets them	Any caller who reaches a service directly can forge them
Forward the user's access token	Services validate it themselves	Token with broad audience flows everywhere
Internal signed token	Gateway mints a short-lived, internally signed token for each request	Requires internal key management, but services can verify
Token exchange (RFC 8693)	Exchange the user token for a narrower, service-specific one	More moving parts

INDUSTRY PRACTICE

Netflix has publicly described (in its tech blog) moving authentication to the edge and passing an internal, signed identity structure to downstream services, so that services do not each parse external tokens. The broader pattern of edge authentication plus internally verifiable identity is common at large companies.

THE INTERNAL NETWORK TRAP

"Services trust any request from inside the VPC" means one SSRF bug or compromised pod becomes full access to every service. Services should verify identity on every call (Lessons 6.2 and 6.3).

Takeaways

- Edge authentication, service-level authorization.
- Propagate identity in a form services can verify, not in forgeable headers.

Assessment

SCENARIO 1 · FORGED HEADER

The payments service trusts `X-User-Id` from the gateway. A developer exposes payments on a debug port. Describe the attack and two layers of fixes.

SCENARIO 2 · FAN-OUT

A request to `/checkout` fans out to six services, three of which call others. Design how the user's identity and the calling service's identity travel through the whole chain.

LESSON 6.2

Service-to-Service Identity

Machines need identities too, and they must not be shared secrets copied into config files.

Learning objectives

- Compare API keys, client credentials, mTLS and workload identity.
- Understand SPIFFE and service meshes.
- Eliminate long-lived static credentials.

The options

Mechanism	How it works	Strengths	Weaknesses
Static API key	Shared string in env vars	Simple	Leaks, never rotated, no identity granularity
OAuth client credentials	Service gets a token from the auth server with its client ID/secret	Standard, scoped, short-lived tokens	Still a secret to protect
Mutual TLS (mTLS)	Both sides present certificates	Strong, at transport level	Certificate lifecycle management
Workload identity	Platform attests the workload and issues short-lived credentials	No stored secrets	Platform-specific setup

Workload identity is the modern direction: the platform itself vouches for the workload. AWS IAM roles for EC2/ECS/EKS, Google Cloud workload identity, Azure managed identities, and GitHub Actions OIDC tokens (letting CI deploy to a cloud without stored keys) all remove long-lived secrets.

SPIFFE/SPIRE (CNCF graduated) standardizes workload identities like `spiffe://shop.com/ns/prod/sa/payments` delivered as short-lived X.509 or JWT credentials. **Service meshes** (Istio, Linkerd) automate mTLS between services so every connection is authenticated and encrypted, and allow policies such as "only `checkout` may call `payments`."

```
checkout pod ── mTLS (cert: spiffe://shop.com/sa/checkout) ──► payments pod
                                                              policy: allow from sa/checkout only
+ application layer: user identity token from the gateway ("on behalf of user 88")
```

KEY IDEA

A service-to-service call has two identities: the calling service (who is making the call) and, often, the user on whose behalf it acts. Robust systems verify both.

Takeaways

- Prefer platform-issued, short-lived workload credentials over static secrets.

- mTLS authenticates services; tokens carry user context; policies decide.

Assessment

SCENARIO 1 · THE LEAKED KEY

An internal API key used by eight services is found in a public GitHub repository. Plan the next 24 hours and the long-term fix.

SCENARIO 2 · CI DEPLOY

ShopSphere's GitHub Actions workflow deploys to AWS using an access key stored in repository secrets. Propose a keyless design and explain its security benefits.

LESSON 6.3

Zero Trust

Never trust the network; verify every request, user and device.

Learning objectives

- Explain zero trust principles and their origin.
- Distinguish zero trust as architecture from zero trust as a product label.
- Apply it to ShopSphere's internal tools.

From castle to checkpoints

Traditional security assumed a hard perimeter (VPN, firewall) and a soft, trusted inside. Attackers who got inside (phishing, a compromised laptop) could move freely. Google's **BeyondCorp** papers, published from 2014 onward, describe how Google moved access decisions away from network location to the **identity of the user and the state of the device**, for every request, allowing employees to work from untrusted networks without a traditional VPN. NIST SP 800-207 later formalized "Zero Trust Architecture."

Principles

- No implicit trust from network location.
- Authenticate and authorize every request, using user identity, device posture and context.
- Least privilege, short-lived access.
- Continuous evaluation and logging.

```
Employee laptop ──► Identity-aware proxy ──► policy: user in "support" group?
                    (e.g. corporate SSO)          device managed & patched?
                                                  MFA fresh? risk low?
                                              ──► allow to admin dashboard (this request only)
```

INDUSTRY PRACTICE

Google Cloud's Identity-Aware Proxy, Cloudflare Access, Zscaler, Microsoft Entra Conditional Access and similar products implement these ideas for workforce access. "Zero trust" is also heavily used in marketing; judge products by whether they actually make per-request, identity-based decisions.

Takeaways

- Zero trust replaces network location with identity, device and context, evaluated per request.
- It applies to users *and* services (mTLS, workload identity).

Assessment

SCENARIO 1 · ADMIN DASHBOARD

ShopSphere's admin dashboard is only reachable via VPN, with a shared admin password. Redesign access using zero trust principles.

LESSON 6.4

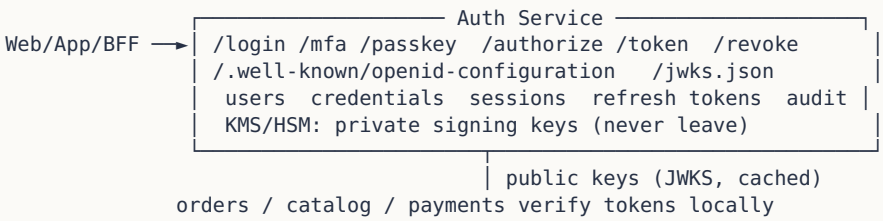
Designing an Auth Service

Keys, JWKS, rotation, sessions and scaling: what the identity core of a company looks like.

Learning objectives

- Design the components of a central authentication service.
- Manage signing keys and rotate them without downtime.
- Plan for availability, since auth outages are total outages.

Components



Key rotation without downtime

1. Generate key `k2027` and **publish** it in the JWKS alongside `k2026` (do not sign with it yet).
2. Wait longer than the maximum JWKS cache time of verifiers.
3. **Switch signing** to `k2027`.
4. Wait longer than the maximum token lifetime.
5. **Remove** `k2026` from the JWKS.

Verifiers select keys by `kid` and refresh the JWKS on an unknown `kid` (with rate limiting, so attackers cannot force constant refetching). For emergency rotation after a key compromise, you skip the waits and accept that users must re-authenticate.

Availability

When auth is down, everything is down. Local token verification (cached JWKS) means APIs keep working even if the auth service is briefly unavailable, while logins and refreshes fail. Session stores need replication. Rate-limit the auth service's own expensive operations (password hashing) so login floods do not take down token refresh.

Build vs buy at this layer

This is exactly where managed identity platforms earn their cost (Phase 8). Building this well requires cryptography, standards compliance, scale and on-call ownership. Many companies run Keycloak (open source) or buy Auth0/Okta, Cognito, Entra External ID, Firebase Auth, Clerk or similar.

Takeaways

- Private keys in KMS/HSM; public keys via JWKS; rotate with overlap.
- Design for auth availability, because auth failure is product failure.

Assessment

SCENARIO 1 · COMPROMISED KEY

The private signing key is found in a log file. Walk through the incident response, including what happens to users and services.

SCENARIO 2 · CACHE STAMPEDE

After a deployment, 400 service instances refetch the JWKS simultaneously every few seconds. Explain why this might happen and how to design the verifier.

LESSON 6.5

Revocation, Caching and Consistency at Scale

The CAP theorem shows up in your logout button.

Learning objectives

- Reason about revocation latency as a business requirement.
- Combine caching and revocation without security holes.
- Handle multi-region identity.

The core tension

Fast systems cache. Secure systems need current truth. Every authorization cache is a window where a revoked permission still works. The right window is a **business decision**: shopping-cart access can lag a minute; a fired employee's access to customer data should not.

Mechanism	Revocation latency	Cost
Access token TTL only	Up to TTL (e.g. 5–15 min)	Free
Denylist pushed via pub/sub to services	Seconds	Infrastructure, must be reliable
Per-request session check	Immediate	Latency, central dependency
Continuous access evaluation (event-driven)	Near-real-time	Complex; standards like the OpenID Shared Signals Framework / CAEP aim to standardize it

```
Admin bans user 88 ─► auth service: bump tokenVersion, revoke refresh tokens
                   ─► publish "user.revoked {88, v=7}" ─► all services update local deny cache
Service request: token v=6 < cached v=7 → reject      (window ≈ propagation delay)
```

Multi-region

Sessions and refresh tokens must be readable in every region users hit. Options: global databases with replication (accepting replication lag for revocation), region-pinned users, or stateless access tokens plus replicated revocation events. Decide what happens during a partition: fail closed for sensitive operations, fail open (with logging) only where the business explicitly accepts it.

Takeaways

- Revocation latency is a requirement you negotiate, then design for.

- Combine short TTLs with event-driven revocation for critical cases.

Assessment

SCENARIO 1 · NEGOTIATE

Product wants zero-latency revocation; infrastructure wants zero per-request lookups. Facilitate the decision by mapping ShopSphere actions to acceptable revocation windows.

LESSON 6.6

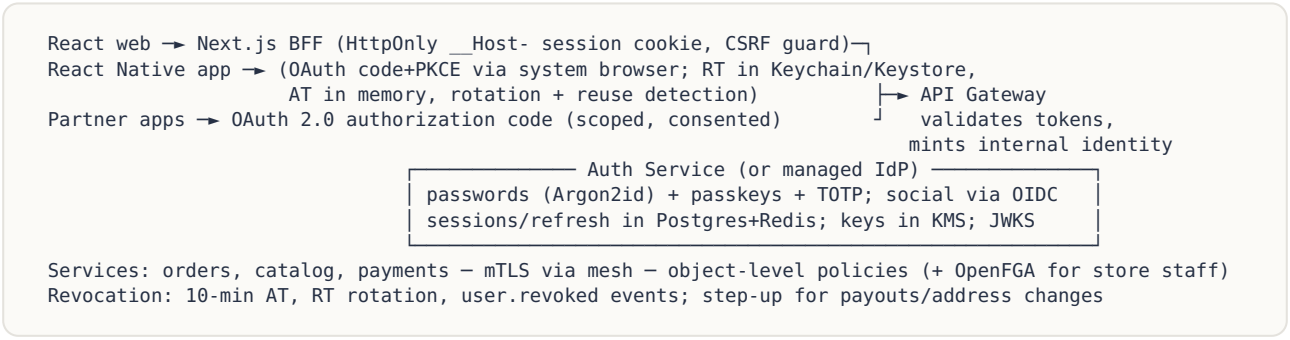
System Design Challenges

Three complete designs that tie the whole course together.

How to approach an auth system design

1. **Clarify** users, clients (web, mobile, partners, services), scale, compliance, risk.
2. **Identify principals and trust boundaries.**
3. **Choose authentication** per client and risk level.
4. **Choose session/token architecture** with revocation requirements.
5. **Choose authorization model(s).**
6. **Address service identity, key management, availability.**
7. **Walk the threat model** and the failure modes.
8. **Decide build vs buy**, with reasons.

Design A: ShopSphere consumer platform (web + mobile)



Key decisions to defend: BFF instead of SPA tokens; passkeys offered to all, required for staff; risk-based step-up; ReBAC only where store staffing demands it; RBAC for internal staff.

Design B: B2B SaaS with enterprise SSO

Tenants resolved by email domain or org subdomain; per-tenant IdP configuration (SAML/OIDC); JIT provisioning plus SCIM for lifecycle; tenant-scoped tokens; RLS in PostgreSQL; per-tenant roles plus custom roles for enterprise tier; audit log export; admin consent for third-party integrations; strong support-impersonation controls.

Design C: Global, multi-region platform

Regional auth endpoints with globally replicated user and credential data; local token verification with JWKS caching; replicated revocation events; region-aware data residency (some tenants' identity data must stay in-region); graceful degradation when the primary region fails (existing sessions keep working, new logins may require region failover).

Assessment

CHALLENGE 1 · RIDE-SHARING

Design authentication and authorization for a ride-sharing platform with rider app, driver app, web dashboard for fleet operators, and internal support tools. Cover driver identity verification, location data access, and support access to trips.

CHALLENGE 2 · FOOD DELIVERY PARTNER API

Restaurants use a third-party POS system that must create menu items and receive orders via your API. Design the integration's identity and authorization, including revocation and abuse limits.

CHALLENGE 3 · CRITIQUE

A design proposes: one JWT signed with HS256 shared across all services, 7-day lifetime, roles inside the token, stored in localStorage on web and AsyncStorage on mobile. List every issue you find, ranked by severity, and propose fixes.

PHASE 7

Identity Management and the Enterprise

Identity has a lifecycle beyond login: provisioning, deprovisioning, workforce and cloud IAM, machine secrets, and the audit trails that compliance and incident response depend on.

IN THIS PHASE

1. Identity Lifecycle and SCIM
2. Workforce IAM and Cloud IAM
3. API Keys and Secrets Management
4. Audit Logging and Compliance

LESSON 7.1

Identity Lifecycle and SCIM

Joiners, movers and leavers: identity is a process, not a row in a table.

Learning objectives

- Explain joiner-mover-leaver processes and why deprovisioning matters most.
- Understand JIT provisioning and SCIM.
- Design for orphaned and dormant accounts.

The lifecycle

Joiner: account created, initial access granted. **Mover:** changes team, needs new access *and removal of old access* (privilege creep happens when only the first part occurs). **Leaver:** all access removed promptly, including sessions, tokens, API keys and OAuth grants they created.

Provisioning methods

- **Manual**: admins create users. Error-prone; leavers are forgotten.
- **Just-in-time (JIT)**: account created on first SSO login from IdP attributes. Easy, but it never *removes* anyone.
- **SCIM 2.0** (RFC 7643/7644): the customer's IdP pushes creates, updates and deactivations to your app over a standard REST API.

```
PATCH /scim/v2/Users/2819c223 HTTP/1.1
Authorization: Bearer <tenant-scoped SCIM token>
Content-Type: application/scim+json

{ "schemas": ["urn:ietf:params:scim:api:messages:2.0:PatchOp"],
  "Operations": [{ "op": "replace", "path": "active", "value": false }] }
```

When `active` becomes false, your app must revoke sessions and tokens immediately, not just block the next login.

> **INDUSTRY PRACTICE**
>
> Okta, Microsoft Entra ID, Google Workspace and OneLogin act as SCIM clients. Enterprise buyers commonly treat SCIM as a requirement alongside SSO, because it makes offboarding automatic.

Takeaways

- Deprovisioning is the security-critical step; JIT alone never does it.
- SCIM deactivation must trigger session and token revocation.

Assessment

SCENARIO 1 · THE CONTRACTOR

A contractor's SSO account is disabled, but they had created a personal API key in ShopSphere and authorized a third-party integration. What still works, and what must your deprovisioning logic do?

LESSON 7.2

Workforce IAM and Cloud IAM

Identities for employees, and the permission systems of the cloud you deploy to.

Learning objectives

- Distinguish customer identity (CIAM) from workforce identity.
- Read and write cloud IAM policies with least privilege.
- Understand privileged access and break-glass procedures.

CIAM vs workforce IAM

	Customer identity (CIAM)	Workforce identity
Users	Millions, self-registering	Thousands, managed by IT
Priorities	Conversion, low friction, scale, privacy	Control, compliance, offboarding
Typical products	Auth0, Cognito, Firebase Auth, Clerk, Entra External ID	Okta Workforce, Microsoft Entra ID, Google Workspace

Cloud IAM

Cloud platforms evaluate policies on every API call. In AWS, an identity policy might be:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject"],
    "Resource": "arn:aws:s3:::shopsphere-invoices/*",
    "Condition": { "StringEquals": { "aws:PrincipalTag/team": "billing" } }
  }]
}
```

AWS evaluation follows "explicit deny overrides allow; default is deny." Misconfigured cloud IAM (wildcard actions, public buckets, over-privileged roles) is behind many publicly reported data exposures.

Privileged access

Admin access should be **just-in-time** (requested, approved, time-boxed), phishing-resistant (hardware keys), logged, and reviewed. Keep **break-glass** accounts for emergencies, heavily monitored and rarely used.

Takeaways

- Customers and employees need different identity systems and priorities.
- Cloud IAM is authorization for your infrastructure; least privilege applies there too.

Assessment

SCENARIO 1 · REVIEW THE POLICY

A Lambda function's role has `"Action": "s3:*", "Resource": "*"`. It only reads one bucket. Explain the risk and write the least-privilege version.

LESSON 7.3

API Keys and Secrets Management

Long-lived credentials are where many breaches begin.

Learning objectives

- Design API keys that are identifiable, scoped, hashed and rotatable.
- Use secrets managers and avoid secrets in code.
- Detect and respond to leaks.

Designing API keys

- **Prefixed and identifiable**: e.g. `shop_live_` + random bytes. Prefixes let secret scanners (GitHub secret scanning partners, gitleaks) recognize leaks, and let you tell test from live keys. GitHub publicly documented adopting identifiable token prefixes for this reason.
- **Store only a hash**; show the key once.
- **Scoped** to specific permissions and, optionally, IP ranges.
- **Owned** by a user or service account, with `lastUsedAt`, expiry and easy rotation (allow two active keys during rotation).

```
function createApiKey() {
  const secret = crypto.randomBytes(32).toString("base64url");
  const key = `shop_live_${secret}`;
  return { key, hash: sha256(key), last4: key.slice(-4) };   // store hash + last4 only
}
```

Secrets management

Application secrets (DB passwords, signing keys, third-party API keys) belong in a secrets manager: AWS Secrets Manager, Google Secret Manager, Azure Key Vault, HashiCorp Vault. Prefer **dynamic, short-lived secrets** where possible. Keep `.env` files out of version control, scan repositories and CI logs, and never bake secrets into Docker images or mobile app bundles.

> **MOBILE (REACT NATIVE)**
>
> Any key in a mobile app (maps, analytics, payments publishable keys) is public. Use only keys designed to be public, restrict them by app identifier/bundle ID in the provider console, and route privileged calls through your backend.

Takeaways

- API keys: prefixed, hashed, scoped, owned, expiring, rotatable.
- Secrets in managers, not code; assume leaks and plan detection and rotation.

Assessment

SCENARIO 1 · PARTNER KEYS

ShopSphere's partner API keys never expire, are stored in plaintext, and have full account access. Design a migration to a better scheme without breaking 400 partners.

LESSON 7.4

Audit Logging and Compliance

If it isn't logged, it didn't happen, and you can't prove it didn't.

Learning objectives

- Decide what to log for authentication and authorization events.
- Protect logs from tampering and from leaking secrets.
- Understand what SOC 2, ISO 27001 and GDPR imply for engineers.

What to log

Who (principal and acting-on-behalf-of), what (action, resource), when, where (IP, device, user agent), outcome (allowed/denied, reason), and a correlation ID. Key events: logins (success and failure), MFA changes, password resets, role and permission changes, token issuance and revocation, admin actions, impersonation, data exports.

Never log: passwords, full tokens, session IDs, API keys, MFA secrets, full card numbers. Log token `jti` or a hash instead.

Protecting logs

Write-once or append-only storage, restricted access, retention policies, and forwarding to a SIEM for alerting (e.g. "refund_reuse_detected," "admin login from new country").

Compliance, from an engineer's perspective

- **SOC 2 / ISO 27001**: auditors look for access controls, MFA for staff, access reviews, offboarding evidence, logging and change management.
- **GDPR / India's DPDP Act**: lawful basis for data, minimization, user rights (access, deletion), breach notification. Identity data (emails, phone numbers, IP addresses) is personal data.
- **PCI DSS**: if you touch card data, strict authentication requirements apply; most companies avoid the scope by using a payment provider's hosted fields.

Takeaways

- Log security events richly and secrets never.
- Compliance frameworks mostly ask you to prove you do the things this book teaches.

Assessment

SCENARIO 1 · THE DISPUTED REFUND

A manager claims they never issued a ₹4,00,000 refund. What evidence should your systems be able to produce, and what does it say about your audit design?

PHASE 8

The Industry: Build vs Buy

How real companies choose between writing auth themselves, using libraries, and adopting managed identity platforms, and what the trade-offs look like over years rather than sprints.

IN THIS PHASE

1. The Landscape
2. The Decision Framework

LESSON 8.1

The Landscape

A map of the tools teams actually use, and what each is good at.

Learning objectives

- Categorize identity tools and know representative options in each.
- Understand typical choices at different company stages.

Categories

Category	Examples	You own
Primitives / libraries	`argon2`, `jose`, `openid-client`, SimpleWebAuthn, Passport.js	Everything above the primitive
Auth frameworks (self-hosted, in your app)	Better Auth, Auth.js (NextAuth), Lucia (now a learning resource), Supabase Auth (self-hosted mode)	Hosting, data, upgrades
Self-hosted identity servers	Keycloak, Ory (Kratos/Hydra), Authentik, ZITADEL	Operations, scaling, security patching
Managed CIAM	Auth0 (Okta), AWS Cognito, Firebase Auth / Google Identity Platform, Microsoft Entra External ID, Clerk, Stytch, Descope, Supabase Auth	Integration and configuration
Workforce IdPs	Okta, Microsoft Entra ID, Google Workspace, JumpCloud	Policies
Enterprise SSO for SaaS	WorkOS, Auth0, BoxyHQ (open source)	Integration
Authorization services	OpenFGA, SpiceDB/AuthZed, Permit.io, Cerbos, OPA, Amazon Verified Permissions	Models and data sync

The JavaScript ecosystem shifts quickly. For example, in September 2025 the Auth.js project announced it was joining Better Auth, with Auth.js continuing to receive security fixes while new projects were pointed to Better Auth. Always check current maintenance status before adopting a library.

Typical patterns by stage

- **Early startup:** managed CIAM or a well-maintained framework; ship quickly with good defaults (MFA, breached-password detection, social login).
- **Growth / B2B:** add enterprise SSO and SCIM (often via a vendor), a real authorization model, audit logs.
- **Large company:** frequently in-house identity platforms built around standards (OIDC, OAuth), sometimes on top of open source, with dedicated identity teams. Google, Microsoft, Amazon and Apple run their own identity providers because identity is core to their products.

Takeaways

- Know the categories; vendors change, categories persist.
- Maintenance status and standards compliance matter more than features lists.

Assessment

SCENARIO 1 · PICK FOR A STAGE

Recommend an identity approach for (a) a two-person startup building a React Native app, (b) a 40-person B2B SaaS with its first enterprise customers, and (c) a bank's new mobile app. Justify each.

LESSON 8.2

The Decision Framework

Build what differentiates you; buy what only hurts you when it breaks.

Learning objectives

- Evaluate build vs buy with a repeatable framework.
- Understand lock-in, migration and total cost.

Questions to ask

1. **Is identity a differentiator?** For most companies, no. For identity companies and very large platforms, yes.
2. **What are the requirements over 3 years?** SSO, SCIM, passkeys, MFA, B2B tenancy, compliance, data residency.
3. **Team capacity:** who is on call for auth at 3 a.m.? Who tracks new attacks and standards?
4. **Cost model:** managed pricing per monthly active user can become significant at consumer scale; self-hosting costs engineers.
5. **Lock-in and exit:** can you export password hashes (and in what format)? Are tokens standard OIDC? Can you migrate users lazily?
6. **Data residency and privacy** requirements.
7. **Customization:** does the product support your flows (e.g. phone-first login, custom risk rules)?

Factor	Build	Library/framework	Managed platform
Time to secure launch	Slow	Medium	Fast
Control	Total	High	Limited to configuration
Security expertise required	Highest	High	Lower (but configuration still matters)
Cost at scale	Engineering salaries	Engineering + hosting	Per-user pricing
Lock-in	None	Low	Medium to high

KEY IDEA

"Buy" does not mean "no thinking." Most breaches in managed-platform deployments come from integration mistakes: broken authorization in your API, tokens stored badly, misconfigured callbacks. Everything in Phases 1–7 still applies.

Migration patterns

Lazy migration: on each user's next login, verify against the old system and write credentials to the new one. **Bulk import** of password hashes if the target supports the algorithm. Run both systems in parallel with a router during transition.

Assessment

SCENARIO 1 · LEAVING A VENDOR

ShopSphere's managed auth bill will exceed the cost of three engineers next year at 20 million MAU. The CFO wants to self-host. Present the full picture to leadership, including risks, migration plan and the conditions under which you would recommend staying.

PHASE 9

Breaking and Defending

Thinking like an attacker, testing like a professional, responding like an incident commander, and proving your skills in a capstone.

IN THIS PHASE

1. Auth in the OWASP Top 10 and API Security Top 10
2. Testing Authentication and Authorization
3. Incident Response
4. Capstone

LESSON 9.1

Auth in the OWASP Top 10 and API Security Top 10

The industry's shared list of what goes wrong, mapped to what you now know.

OWASP Top 10:2025 and this book

2025 category	Where it lives in auth	Lessons
A01 Broken Access Control (now includes SSRF)	IDOR/BOLA, missing role checks, CORS misconfiguration, forced browsing	1.1, 5.1–5.6
A02 Security Misconfiguration	Debug endpoints, default credentials, permissive CORS, cookie flags	2.3, 6.1
A03 Software Supply Chain Failures	Compromised auth dependencies, abandoned libraries	8.1
A04 Cryptographic Failures	Weak hashing, weak JWT secrets, no TLS	1.3, 1.4, 2.4
A05 Injection	SQL injection in login, XSS stealing tokens	2.6
A06 Insecure Design	Recovery bypassing MFA, missing threat model	1.5, 2.8
A07 Authentication Failures	Stuffing, weak sessions, enumeration, fixation	Phases 2–3
A08 Software or Data Integrity Failures	Unsigned tokens or updates, deserialization	2.4
A09 Security Logging and Alerting Failures	No auth audit trail, no alerting	7.4
A10 Mishandling of Exceptional Conditions	Failing open when auth dependencies error	2.2, 5.3, 6.5

The **OWASP API Security Top 10 (2023)** is even more auth-heavy: Broken Object Level Authorization (API1), Broken Authentication (API2), Broken Object Property Level Authorization (API3), and Broken Function Level Authorization (API5) are all access-control or authentication issues.

KEY IDEA

A10:2025 deserves special attention for auth: code like `try { verify(token) } catch { next() }` silently turns an error into access. Auth code must fail closed.

Assessment

SCENARIO 1 · FAIL OPEN

Find the bug: `catch (e) { logger.warn(e); return next(); }` in authentication middleware. When would it trigger in production, and what is the correct behavior?

LESSON 9.2

Testing Authentication and Authorization

Professional techniques for finding the bugs before someone else does.

The toolkit

- **Intercepting proxies:** Burp Suite (Community/Pro), OWASP ZAP, Caido for web; Proxyman, Charles or mitmproxy for mobile (install the proxy's CA certificate on a test device; apps with pinning need a debug build).
- **Autorize-style testing:** replay every request with a second user's and an unauthenticated session.
- **JWT tooling:** jwt.io for decoding; Burp's JWT extensions for alg/kid tampering tests.
- **Automated tests:** authorization matrices in your test suite; contract tests that every route declares its policy.

A test plan for any app

```
AUTHENTICATION                          AUTHORIZATION
□ Enumeration via login/reset/signup    □ Two-account IDOR on every ID-bearing route
□ Rate limits & lockout behavior        □ Role downgrade: low-priv token on admin routes
□ Session regenerated at login          □ Mass assignment on every write endpoint
□ Logout invalidates server-side        □ Tenant switching / tenant ID tampering
□ Password reset token: entropy, expiry,□ Hidden fields returned by API
  single use, Host header               □ Method switching (GET vs POST vs PUT)
□ JWT: alg none/confusion, aud, exp     □ Cached responses across users
□ MFA bypass: skip step, reuse code     □ OAuth: redirect_uri, state, scope escalation
```

Building it into engineering

Security unit tests for policies; integration tests for every route's 401/403 behavior; dependency scanning; periodic penetration tests; a bug bounty program when mature (many large companies, including Google, Microsoft and GitHub, run public programs).

Assessment

SCENARIO 1 · TEST PLAN

You have two days to security-test ShopSphere's new seller payout feature (web and app). Write your plan, prioritized.

LESSON 9.3

Incident Response

When it goes wrong anyway: containing, investigating and recovering.

The phases

Detect (alerts, reports, bug bounty) → **Contain** (stop the bleeding) → **Eradicate** (remove the cause) → **Recover** (restore normal operation) → **Learn** (blameless post-mortem).

Playbooks

Credential stuffing wave: enable stricter rate limits and bot challenges; force reset for confirmed-compromised accounts; notify affected users; expand breached-password checks and MFA adoption.

Leaked signing key: rotate immediately (emergency path from 6.4); revoke all tokens signed by it; investigate how it leaked; review logs for forged tokens (tokens with unusual claims or from unexpected sources).

Session token theft at scale (e.g. via XSS or a compromised dependency): fix the vector; revoke all sessions; force re-authentication; review access logs for the exposure window.

Insider or compromised admin: disable account and sessions; preserve audit logs; review every action taken; rotate any secrets they could access.

INDUSTRY PRACTICE

Publicly documented incidents are excellent study material: Okta's 2023 support-system breach (session tokens in HAR files uploaded to support), the 2022 Uber MFA-fatigue breach, and many cloud IAM misconfigurations. Read the companies' own post-incident reports and ask, for each step, which control from this book would have stopped it.

Assessment

SCENARIO 1 · HAR FILES

ShopSphere's support team asks customers to upload browser HAR files to debug issues. Explain the risk using what you know about sessions and tokens, and design safer support tooling.

LESSON 9.4

Capstone

Build it, break it, review it, defend it.

The project

Build **ShopSphere Lite**: a Node.js API, a React web app and a React Native app with:

1. Email/password login with Argon2id, enumeration resistance and rate limiting.
2. Web: BFF with HttpOnly `__Host-` session cookies and CSRF protection.
3. Mobile: tokens with refresh rotation and reuse detection, stored in secure storage.
4. TOTP MFA with recovery codes, plus passkeys on web.
5. "Sign in with Google" via OIDC with safe account linking.
6. Roles (customer, seller, support, manager) plus ownership checks, centralized in a policy module.
7. Step-up authentication for refunds and payout changes.
8. Password reset, email change, "log out everywhere."
9. An audit log of security events.
10. An authorization matrix test suite.

Then

- **Break it:** run the test plan from 9.2 against your own system. Document every finding.
- **Review it:** write a one-page threat model and a list of known weaknesses.
- **Defend it:** write a design document explaining every major decision and its alternatives, as if presenting to a senior engineer. Bring it to Claude for a mock design review.

> **YOU ARE DONE WHEN**
>
> **You can look at unfamiliar authentication code or an architecture diagram and, within minutes, say where the trust boundaries are, how identity travels, where authorization is enforced, what would happen if each credential leaked, and what you would change first.**

Appendix A: Answer Guide

These are the points a strong answer covers. Write your own answer first; then compare. If your reasoning reached the same conclusions by a different path, that is fine. If you missed a point, re-read the lesson section that covers it. For anything you disagree with or want to go deeper on, bring your answer to Claude for a review.

Phase 1

Lesson 1.1

1 · Hidden button. Hiding UI is not authorization. Through the dashboard: open DevTools, flip `currentUser.role` or call the API from the console. Without any client: `curl -X DELETE` with the agent's own valid credential. Fix on the server: a function-level permission check (`sellers:delete`) on the endpoint, deny by default, audit log; keep the hidden button purely for UX.

2 · Secret in the app. It is neither user nor reliable app authentication: the key is extractable from the bundle or intercepted traffic within hours, then usable from any script. It only stops casual scraping. Better: real user authentication for user actions, rate limiting and bot detection for abuse, and app attestation (App Attest, Play Integrity) as a risk signal if "genuine app" matters.

3 · Classify. (a) AuthN: wrong principal accepted via reused credentials. (b) AuthZ: correct principal, missing object-level check. (c) AuthZ: missing function-level check. (d) AuthN: the system believes the attacker is the victim (stolen bearer credential), though session hygiene (expiry, binding) is the defense.

4 · 403 vs 404. 404 hides whether the resource exists (prevents enumeration and confirming that order 9812 is real). Downsides: harder debugging, confusing for legitimate users who lost access (e.g. removed from a team). For random UUID order IDs either is acceptable; many choose 404 for other users' private objects. Where existence itself is sensitive (a user's medical record, a private repository), 404 is clearly better.

5 · Stale proof. Yes, it is a security bug. The sessions are proofs from an authentication event that is no longer trustworthy. On password change (especially for suspected compromise), revoke all other sessions and refresh tokens, optionally keep the current one, and notify the user. Trade-off: users are logged out on other devices; revocation requires server-side state or short-lived tokens.

Lesson 1.2

1 · Session in URL. Leaks via browser history, server/proxy/CDN logs, analytics tools, `Referer` headers to third-party resources, screenshots and copied links shared with others (instant account sharing). Alternative: HttpOnly cookie; if cookies are truly unavailable, an `Authorization` header with in-memory storage.

2 · One API, two clients. Web: HttpOnly session cookie via BFF (protected from XSS theft, needs CSRF defense). Mobile: bearer token in Keychain/Keystore sent in a header. Same mechanism for both is not ideal because cookies suit browsers' same-site model and mobile cookie jars are inconsistent, while bearer tokens in browsers expose them to JavaScript.

3 · Logged out locally. `Secure` cookie on `http://localhost` (not sent or not stored, depending on browser); missing `credentials: 'include'` on cross-origin `fetch` (ports differ = different origins); CORS not allowing credentials or using `*`; `SameSite` or `Domain` misconfiguration. Fix with a dev proxy so frontend and API share an origin.

Lesson 1.3

1 · Legacy MD5. Wrap immediately: compute `Argon2id(md5_hash)` for every row now and mark them `legacy_wrapped`. At login, verify with `Argon2id(md5(input))`; on success, rehash as `Argon2id(input)` directly. No plaintext needed, and every stored hash becomes slow and salted on day one.

2 · Frontend hash. It hides the raw password from your server logs, but the hash *becomes the password*: whoever captures it can replay it. Unsalted SHA-256 also allows cross-site correlation. TLS already protects transit; hash slowly on the server. (Advanced protocols like OPAQUE exist, but that is not what this is.)

3 · Raising the cost. Trade-offs: stronger offline resistance vs latency and CPU. Mitigate DoS: strict rate limiting before hashing, bot defenses, a bounded worker pool for hashing, separate capacity for login vs other traffic, and tune parameters to a measured latency budget.

Lesson 1.4

1 · Ten microservices with HS256. The compromised service holds the signing secret, so the attacker can mint valid tokens for any user, including admins, accepted by all ten services. Use asymmetric signing (EdDSA/ES256): only the auth service holds the private key; services get public keys via JWKS. Also validate `aud` per service.

2 · Base64 "encryption". Problem one: anyone can read it (no confidentiality). Problem two, worse: anyone can modify it (no integrity), so users can set `role: admin`. Minimum fix: store only an opaque session ID, or at least sign (HMAC) the cookie, and never trust client-held roles without verification.

3 · Pinning. Addresses interception by attackers who can get a rogue certificate trusted (hostile networks, compromised CAs, some enterprise proxies). Risk: rotation mistakes brick the app for all users until an update. Mitigate: pin public keys (SPKI) not certificates, include backup pins, have a remote kill switch or graceful fallback, monitor expiry.

Lesson 1.5

1 · SMS login. Spoofing: SIM swap, SMS interception. Tampering: changing the phone number in the verify request. Repudiation: no log of which number received which code. Information disclosure: responses revealing registered numbers. DoS: SMS bombing; SMS pumping costs. Elevation: OTP verification that does not bind code to the phone number, letting one code log into another account.

2 · Prioritize. (b) first: missing admin role checks enable direct privilege escalation. Then (d) or (c): (c) CSRF exposure depends on other defenses; (d) is critical for investigations and compliance. (a) enumeration last: real but lower impact. A strong answer explains likelihood × impact rather than listing.

Phase 2

Lesson 2.1

1 · Lockout weapon. Replace hard lockout with progressive delays and CAPTCHA/step-up per account; never lock out successful logins from known devices; rate-limit by IP/device fingerprint as well; alert on targeted patterns; encourage passkeys/MFA for sellers so password guessing becomes pointless.

2 · Timing oracle. The 600 ms vs 20 ms difference reveals registered emails. Fix: enqueue the email asynchronously and respond immediately in both cases; optionally add jitter. Same principle: equalize side effects, not just messages.

3 · Stuffing in progress. Distributed credential stuffing. Next hour: tighten global and per-account limits, enable bot challenges, block obvious ASNs/proxies, watch success rate, force resets for accounts that succeeded from suspicious sources. Next week: breached-password checks, MFA/passkey push, risk scoring, better alerting on failure-ratio anomalies.

Lesson 2.2

1 · Scaling sessions. In-memory sessions live on one server; the load balancer routes requests elsewhere. Sticky sessions are fragile (redeploys, autoscaling lose sessions). Choose a shared store (Redis with replication) so any instance can validate.

2 · Password changed. Per-user index of session hashes. On change: iterate the set, delete all except the current session's hash; revoke refresh-token families; bump token version for JWTs; notify the user.

3 · Redis outage. Fail closed for authentication (failing open means anyone is anyone). Reduce blast radius: replicated/managed Redis with failover, sessions persisted in Postgres with Redis as cache, short local caches of recently validated sessions (accepting brief revocation delay), and graceful degradation for anonymous browsing.

Lesson 2.3

1 · Compromised subdomain. `blog.shop.com` is same-site, so SameSite does not block its requests, and with `Domain=shop.com` the cookie is also sent to (and settable from) the blog. The attacker can perform CSRF-style actions and potentially overwrite cookies (cookie tossing). Fixes: host-only `__Host-` cookies (no Domain), and origin checks requiring `same-origin` plus CSRF tokens; move untrusted content to a separate registrable domain.

2 · GET logout. An attacker can log users out at will with an `<img>` tag (nuisance, and enables login CSRF chains). `GET /cart/add` lets any site modify users' carts. State changes must use POST/PUT/DELETE with CSRF defenses.

3 · Reflected origin. Any website can make credentialed requests to your API and *read* the responses, stealing personal data, CSRF tokens and more from any logged-in visitor. Fix: explicit allowlist of trusted origins.

Lesson 2.4

1 · No options. Without an algorithm allowlist, some libraries/versions may accept HS256 tokens verified with the public key as an HMAC secret (algorithm confusion); also no `iss` / `aud` checks. Fix: `algorithms: ["RS256"]`, issuer, audience, clock tolerance; use a key object bound to the algorithm.

2 · Role in token. The banned seller keeps seller powers for up to 24 hours. Shrink the window: short access-token TTL (5–15 min) with revocable refresh tokens; token version checks or denylists; look up roles server-side for sensitive actions rather than trusting the token.

3 · Audience. The issuer does not set per-service audiences or `payments` does not validate `aud`. Issue tokens with specific `aud` (resource indicators) and make each service reject tokens not meant for it.

Lesson 2.5

1 · Choose. Server-side sessions: instant revocation, simple, fresh data, one Redis or Postgres lookup is trivial at this scale, no token storage problem in the browser. JWTs add rotation, key management and revocation complexity without a matching benefit. Revisit if services multiply.

2 · Two-tab race. Both tabs refresh with the same refresh token; the second looks like reuse and revokes the family. Fixes: a short grace period allowing the immediately previous token (slightly weaker reuse detection); or coordinate refresh across tabs (BroadcastChannel / Web Locks) so only one refreshes; best of all for web, a BFF that refreshes server-side.

3 · Ten-second ban. Short access tokens alone do not meet 10 s. Publish revocation events (user ID + token version) over a reliable bus to all services, which keep an in-memory deny set checked on every request; plus refresh-token revocation. Monitor propagation lag; fail closed on missing revocation feeds for sensitive services.

Lesson 2.6

1 · Reply to the blog post. CSRF is well understood and mitigated (SameSite, origin checks, tokens); token theft via XSS from localStorage is not mitigable once XSS exists and gives long-lived, off-site account takeover. HttpOnly cookies reduce XSS to "acts while the page is open." Prefer cookies (BFF), plus XSS prevention and CSP.

2 · Pure SPA. Authorization code + PKCE, access token in memory only, refresh token with rotation and reuse detection, short lifetimes, strict CSP, minimal third-party scripts, Trusted Types where possible; plan the BFF as soon as possible.

Lesson 2.7

1 · Biometric toggle. AsyncStorage is unencrypted; the biometric check is in JS and bypassable on rooted/jailbroken devices; the token is readable without biometrics at all. Redesign: store the token in Keychain/Keystore with biometric access control so the OS releases it only after biometric success; or use passkeys.

2 · Stolen phone. The thief can use the app while it's unlocked and the session is valid. Server features: device/session list with remote logout from the web, step-up for sensitive actions, short-lived access tokens, notifications for new sensitive actions, and making `WHEN_UNLOCKED` storage the default.

3 · Bot farm. A secret key will be extracted. Use app attestation as a signal, rate limits per device/IP/phone prefix, phone or email verification, risk scoring, and fraud monitoring; make fake accounts less valuable (limits on new accounts).

Lesson 2.8

1 · Host header. The attacker submits a reset request for the victim with `Host: evil.com`. The victim receives a genuine email from ShopSphere containing `https://evil.com/reset?token=…`; clicking it sends the valid token to the attacker, who resets the password. Fix: build links from configured base URLs only.

2 · Support desk. Prefer self-service recovery codes; otherwise require identity proofing (e.g. verifying via a previously verified channel plus account-specific knowledge), a waiting period with notifications to all known channels, and never immediate removal over a phone call. Log and review every manual recovery.

3 · Ten minutes of access. Attacker tries: change email (block: recent re-auth + confirm new + notify old), change password (require current password), add MFA device or passkey (require re-auth), create API keys or OAuth grants (re-auth, notifications), add payout details (step-up). All should trigger notifications.

Phase 3

Lesson 3.1

1 · Pending state. The attacker with the password gets a full session cookie and can skip the MFA page entirely by calling APIs directly. The pre-MFA state must be a separate, limited credential valid only for the MFA endpoint.

2 · Factors. (a) Customers: passkeys where supported (modern Android supports them), SMS/WhatsApp OTP as accessible fallback with fraud controls, TOTP optional; prioritize reach. (b) Staff with refunds: phishing-resistant only (passkeys or hardware security keys), no SMS.

3 · TOTP brute force. One million codes, roughly three valid at a time with drift windows: unthrottled, thousands of requests per second break it in minutes. Limit attempts per code/account (e.g. 5), lock the MFA step (not the account) with backoff, alert, and invalidate the pending login.

Lesson 3.2

1 · Explain to CEO. Your phone creates a special lock-and-key pair for ShopSphere only. The key never leaves the phone and is never typed. When you log in, ShopSphere sends a puzzle that only that key can solve. A fake website is a different address, so your phone refuses to use the ShopSphere key there. There is nothing for a scammer to collect and reuse.

2 · Domain migration. Passkeys are bound to the RP ID; `shopsphere.in` credentials will not work on `shopsphere.com`. Plan: keep the old domain's login working during transition; after users log in, prompt them to create new passkeys on the new domain; consider WebAuthn Related Origin Requests where supported; keep fallback methods active.

3 · Global challenge. Concurrent users overwrite each other's challenges (logins fail randomly), and challenges are not single-use or bound to a session, allowing replay of captured assertions. Store per-attempt, short-TTL, single-use challenges.

Lesson 3.3

1 · Disappearing links. Corporate email scanners pre-fetch links and consume single-use tokens. Make the link land on a confirmation page requiring a POST (button click), or switch to codes, or allow a short multi-use window tied to the same browser.

2 · Login CSRF. The victim, unknowingly logged into the attacker's account, saves addresses, payment methods or browsing data the attacker can later see, or the attacker frames the victim. Binding the link to the requesting browser (a pending cookie) prevents it.

Lesson 3.4

1 · Risk tiers. Browsing: none. Order: existing session, with risk checks on new devices for high-value carts. Address change: recent auth (e.g. within 30 min) or step-up, notify. Payout bank account: phishing-resistant step-up, cooling-off period, notifications to all channels. Staff refund: fresh phishing-resistant auth, plus approval above thresholds.

2 · False positives. Mobile carrier IPs geolocate poorly and change frequently; VPNs and CGNAT make users appear to jump cities. Tune: use device signals over IP geolocation, require multiple signals, use step-up instead of block, whitelist known devices, measure precision.

Phase 4

Lesson 4.1

1 · Password-sharing integration. Sellers lose control (full account, can't scope or revoke, MFA bypass pressure), LedgerLy becomes a juicy breach target, ShopSphere sees "logins" it can't distinguish from attacks. Offer an OAuth-based partner API with scoped, revocable, audited access.

2 · Access token as login. An access token is for the API, not proof the user is present at your app; it could have been obtained by another client and injected (token substitution). Use OIDC: verify an ID token whose `aud` is your client and whose `nonce` matches.

Lesson 4.2

1 · Roles. Resource owner: the seller. Client: LedgerLy. Authorization server: ShopSphere's auth service. Resource server: ShopSphere's orders API. Grant: authorization code with PKCE, `offline_access` for refresh tokens.

2 · Nightly job. Short-lived access tokens, a refresh token (rotated, sender-constrained if possible) stored server-side by LedgerLy. On revocation, the refresh token and grant are invalidated, refresh fails, and the job must stop and notify the seller.

3 · Smart TV. Device authorization grant: TV shows a code and URL/QR; user approves on their phone (already logged in); TV polls for tokens. Avoid typing passwords on a TV remote.

Lesson 4.3

1 · WebView. Your app can read keystrokes/passwords (users should not trust it), it cannot use existing browser sessions (users must log in again), phishing training says "check the URL" but WebViews hide it, and Google blocks OAuth in embedded WebViews (a non-security, functional reason).

2 · Plain method. The challenge equals the verifier, so anyone who sees the authorization request (logs, proxies, malicious apps observing the URL) learns the verifier. S256 protects even if the request is observed.

Lesson 4.4

1 · ID token to API. ID tokens are for the client (`aud` = client ID), not APIs; the API should not accept tokens meant for another audience; Google ID tokens prove Google identity, not ShopSphere permissions. Correct: backend verifies the ID token once at login, then issues ShopSphere's own session or access tokens with ShopSphere audiences.

2 · Email linking. Attacker creates a GitHub account with the victim's email (if unverified or from a provider with weaker verification) and signs in, getting linked to the victim's ShopSphere account. Safe design: link by `iss` + `sub`; auto-link only with verified emails from trusted providers; otherwise require logging in to the existing account first.

Lesson 4.5

1 · Scopes. `orders:read`, `orders:write` (fulfillment status), `products:read`, `products:write`, `inventory:read`, `inventory:write`, `payouts:read`. Never to third parties: payout destination changes, account security settings, customer PII bulk export, admin functions.

2 · Scope-only check. BOLA: any client with `orders:read` from any seller can read any order by ID. Check the user's ownership too (intersection).

Lesson 4.6

1 · Offboarding. Shorter session lifetimes for SSO tenants, periodic re-validation with the IdP, and SCIM deactivation events that revoke sessions and tokens immediately. Back-channel logout (OIDC) can also help.

2 · Build or buy. Six weeks, two engineers, three different enterprise IdPs: buy (WorkOS, Auth0, Keycloak if you can operate it). SAML's security pitfalls and per-IdP quirks make building risky on that timeline; keep your user model standard to avoid lock-in.

Lesson 4.7

1 · Wildcard redirect. Any subdomain takeover, user-content subdomain, or open redirector on any subdomain can steal codes. Alternatives: register each exact redirect URI, use one central callback that routes internally, or dynamic client registration per subdomain.

2 · Callback review. No `state` check (CSRF); no PKCE; tokens in localStorage (XSS theft); open redirect via unvalidated `next`; possibly no `nonce`/ID token validation; code remaining in browser history (should clear URL).

Phase 5

Lesson 5.1

1 · Mass assignment. Users can set `role`, `emailVerified`, `balance`, `tenantId`. Fixes: allowlist fields per endpoint (`pick`); or validate with a strict schema (e.g. zod with `.strict()`) that rejects unknown fields.

2 · Make it unforgettable. Central policy module; default-deny router requiring every route to declare a policy (fail the build otherwise); repository methods that require a principal and scope queries; authorization matrix tests in CI; code review checklist; periodic automated two-account scans.

Lesson 5.2

1 · 87 roles. Roles encoding data (region, amount, store) multiplied. Migration: map roles to permissions; introduce attributes/relationships for the data dimensions; build a compatibility layer; migrate one feature at a time with shadow evaluation (compute old and new decisions, log differences) before switching.

Lesson 5.3

1 · Policies. Support: allow `order:view`; deny fields `payment.*` (property-level filter). Manager refund: allow if region matches and amount < 50,000. Above: create a pending refund requiring approval from a *different* manager or finance (separation of duties).

2 · PDP down. Checkout should not depend on a remote call per request: embed the policy engine as a library or sidecar with locally cached policies and data; fail closed for sensitive actions, with clear errors; monitor and alert.

Lesson 5.4

1 · Model. `store:S#owner@user:A`, `store:S#manager@user:B`, `store:S#catalog_editor@user:C`, `product:P#store@store:S`. Rule: editor of product = owner, manager or catalog_editor of its store. Check: `can user:X edit product:Y`.

2 · New enemy. The permission check used a replica or cache that had not yet seen the removal. Zanzibar's consistency tokens ensure checks for content are evaluated at least as fresh as the latest ACL change relevant to it.

Lesson 5.5

1 · Cache leak. Cache key lacked tenant (or user) context, so one tenant's response served another. Fix: include tenant in every cache key by construction (a tenant-aware cache wrapper), and treat caches as part of authorization reviews.

2 · Impersonation. Requires specific permission, a reason/ticket, customer consent for sensitive tenants, time-boxed session, visible banner, read-only by default, separate token marking `act` (actor) vs `sub`, blocked from security settings and payouts, full audit log visible to the customer.

Lesson 5.6

1 · Hidden fields. The phone number is in the API response and the DOM; anyone can read it in DevTools. The API must omit or mask the field for roles not allowed to see it.

2 · Stale permissions. Not a security bug if the server denies; it is a UX bug. On tap, the API returns 403; the app should refresh permissions and explain. Improve by refreshing on foreground and pushing permission-change events.

Phases 6–9

Lesson 6.1

1 · Forged header. Anyone reaching the debug port sends `X-User-Id: <any user>` and acts as them. Fixes: never expose internal ports; services authenticate callers (mTLS) and verify an internally signed identity token rather than plain headers.

2 · Fan-out. Service identity via mTLS (mesh) on every hop; user context as a short-lived signed internal token (or exchanged token, RFC 8693) with narrowed audience per downstream; services authorize both "may this service call me?" and "may this user do this?"

Lesson 6.2

1 · Leaked key. Immediately rotate (issue new key, deploy to all eight services, revoke old), review logs for use of the old key, assess data accessed, rotate anything reachable from it. Long term: per-service identities (workload identity, mTLS), secrets manager, secret scanning in CI, no shared keys.

2 · Keyless CI. GitHub Actions OIDC: the workflow gets a short-lived OIDC token; AWS trusts GitHub's issuer for a role restricted to the repo/branch/environment; no stored access keys, credentials expire in minutes, and access is scoped per workflow.

Lesson 6.3

1 · Admin dashboard. Remove shared passwords; individual SSO accounts with phishing-resistant MFA; identity-aware proxy evaluating group membership and device posture per request; role-based permissions inside the dashboard; audit logs; short sessions with step-up for dangerous actions.

Lesson 6.4

1 · Compromised key. Treat as critical: generate a new key, publish and switch immediately, remove the leaked key from JWKS (accept forced re-login), revoke refresh tokens if their integrity relied on it, purge the log, find how it got there, hunt for forged tokens, communicate internally.

2 · Cache stampede. Unknown-`kid` refetch logic without rate limiting, or a new key used before publication. Design: cache JWKS with TTL, refetch on unknown `kid` at most once per interval, single-flight per instance, jittered refresh, publish keys before use.

Lesson 6.5

1 · Negotiate. Browsing: TTL-based (minutes). Orders: minutes. Payment method changes, payouts, staff access to customer data, admin actions: seconds via revocation events or per-request checks. Present a table with latency, cost and failure behavior for each.

Lesson 6.6

Challenge 1 · Ride-sharing. Separate rider and driver identities/apps; drivers need identity proofing (documents, background checks, selfie checks) and periodic re-verification; location visible only to matched parties during the trip, with policies expiring access afterward; support access via ticket-scoped, audited, time-boxed permissions; fleet operators as a tenant model.

Challenge 2 · POS partner. OAuth authorization code per restaurant (restaurant admin consents), scopes for menu and orders only, tokens per restaurant (not one global key), webhooks signed with HMAC for orders, rate limits per client, revocation from the restaurant dashboard, sender-constrained tokens if feasible.

Challenge 3 · Critique. Critical: shared HS256 secret (any service can forge); localStorage tokens (XSS theft); 7-day non-revocable tokens; roles frozen in tokens; AsyncStorage unencrypted on mobile. Fix: asymmetric keys with JWKS, short access tokens plus rotating refresh tokens, BFF for web, Keychain/Keystore for mobile, server-side role checks for sensitive actions, `aud` per service.

Lessons 7.1–7.4

7.1 · Contractor. Personal API keys and third-party OAuth grants keep working unless deprovisioning revokes them. Deprovisioning must revoke sessions, refresh tokens, API keys, OAuth grants created by the user, and reassign owned resources.

7.2 · Policy. `s3:*` on `*` lets a compromised function read, delete or make public every bucket. Least privilege: `s3:GetObject` on `arn:aws:s3:::that-bucket/*` (and `s3:ListBucket` on the bucket if needed).

7.3 · Partner keys. Introduce new prefixed, hashed, scoped keys alongside old ones; dashboard for partners to create new keys; track usage of old keys; deadlines with notifications; hash existing keys immediately (store only hashes); eventually expire old keys; offer OAuth for partners acting for sellers.

7.4 · Disputed refund. Immutable audit entry: who (user ID, session ID hash), how they authenticated (method, time, step-up), from which device/IP, the request details, approval chain, timestamps. If you cannot produce that, your logging and step-up design are insufficient.

Lessons 8.1–8.2

8.1 · By stage. (a) Managed CIAM or Better Auth/Supabase for speed with secure defaults. (b) Managed CIAM plus an enterprise SSO vendor (or Keycloak), SCIM, proper RBAC/tenant model. (c) Bank: regulated requirements, likely a managed enterprise platform or in-house on standards, phishing-resistant MFA, device binding, strict audit.

8.2 · Leaving a vendor. Compare full costs (engineers, on-call, security reviews, compliance) vs licensing; capability gaps; migration (password hash export, lazy migration, OIDC compatibility); risk window; recommend staying if hash export is impossible, if the team lacks identity expertise, or if roadmap needs (passkeys, SSO) would stall.

Lessons 9.1–9.3

9.1 · Fail open. Any verification error (expired token, malformed token, JWKS fetch failure) results in `next()` with no user, and downstream code that assumes "if we got here, auth passed" grants access. Correct: return 401 on any verification failure; 503 for infrastructure failures; never continue.

9.2 · Test plan. Day 1: two-account IDOR and role tests on all payout endpoints (web and app via proxy), step-up enforcement at the API, mass assignment on bank details, notifications. Day 2: race conditions (double payout), CSRF on web, token handling in the app, audit logging, edge cases (recently changed password, revoked staff).

9.3 · HAR files. HAR files include cookies and Authorization headers, i.e. live session tokens; anyone accessing the support system can hijack sessions (as in the publicly reported Okta 2023 incident). Safer: sanitize HARs client-side before upload, strip credentials server-side, short session lifetimes, bind sessions to devices where possible, and better diagnostic tooling that never captures credentials.

Appendix B: Practical Projects Ladder

Do these alongside the phases. Each builds on the last and ends up as the Capstone (Lesson 9.4). Use Node.js (Express or Fastify), React (Vite or Next.js) and React Native (Expo is fine).

#	After	Project	Done when
1	1.3	Password hashing CLI: register/verify users in a JSON file with Argon2id; migrate a fake MD5 file	You can explain every field in the stored hash string
2	2.2	Session-based login API with Redis, logout, "log out everywhere," device list	Two-account and no-credential tests pass
3	2.3	React web app on top of #2 with `__Host-` cookies and CSRF guard	A CSRF demo page on another port fails to change data
4	2.4	JWT issuer + verifier with `jose`, EdDSA keys, JWKS endpoint, key rotation script	You can rotate keys with zero failed requests
5	2.5	Refresh token rotation with reuse detection and an audit log	Replaying an old refresh token revokes the family
6	2.7	React Native app: login, secure storage, single-flight refresh interceptor	Ten parallel requests after expiry trigger one refresh
7	3.1	TOTP MFA with recovery codes; pending-MFA state that cannot reach APIs	Skipping the MFA step via API fails
8	3.2	Passkeys on the web app with SimpleWebAuthn	Login works with a platform authenticator
9	4.4	"Sign in with Google" via `openid-client` with safe account linking	Linking requires `email_verified` or explicit login
10	5.1	Central policy module + authorization matrix tests	CI fails if a new route has no declared policy
11	5.4	Model store staff in OpenFGA (local Docker) and use it for product edits	Removing a staff member denies access immediately
12	9.2	Attack your own system with Burp/ZAP and a mobile proxy; write a findings report	You found and fixed at least three real issues

CODE REVIEW DRILL (REPEAT WEEKLY)

Pick an open-source project's auth code (a Node.js app, an Expo app, a library's example) and write a short review: where are the trust boundaries, how is identity carried, where is authorization enforced, what would happen if each credential leaked, and what would you change first? Ask Claude to critique your review.

Appendix C: Security Review Checklist

Use this when reviewing a pull request, a design document or an unfamiliar codebase.

Authentication

- Passwords hashed with Argon2id/bcrypt/scrypt; no plaintext or fast hashes; length-based policy with blocklist.
- Login, signup and reset do not reveal account existence (messages, timing, side effects).
- Rate limiting and bot defenses on every credential-accepting endpoint, including MFA and OTP.
- MFA available; phishing-resistant MFA for staff and high-risk users; pending-MFA state isolated.
- Recovery flows are as strong as primary authentication.

Sessions and tokens

- Session IDs: 128+ bits, regenerated at login, idle and absolute timeouts, server-side logout.
- Cookies: `__Host-`, HttpOnly, Secure, SameSite; no state changes on GET; CSRF defenses for cookie auth.
- JWTs: algorithm allowlist, `iss` / `aud` / `exp` validated, no secrets in payload, asymmetric keys across services, keys in KMS, rotation plan.
- Refresh tokens: rotated, reuse-detected, revocable, hashed at rest.
- Browser: tokens not in localStorage; BFF preferred; CSP present. Mobile: Keychain/Keystore, never AsyncStorage.
- Security events revoke sessions and notify users.

OAuth / OIDC / SAML

- Authorization code + PKCE; exact redirect URIs; `state` and `nonce`; no tokens in URLs.
- ID tokens validated (signature, `iss`, `aud`, `exp`, `nonce`); users keyed by `iss` + `sub`; safe account linking.
- Scopes minimal; audience set and checked; consent for third parties; revocation available.
- SAML via maintained library; signatures, audience, time and replay checked.

Authorization

- Every endpoint has function-level and object-level checks; deny by default.
- Queries scoped by principal and tenant; caches keyed by tenant/user.
- Writable fields allowlisted; sensitive fields omitted from responses for unauthorized roles.
- Frontend checks are UX only; server enforces everything.
- Authorization matrix tests in CI.

Infrastructure and operations

- Services authenticate each other (mTLS/workload identity); no trusted-network assumptions; no forgeable identity headers.
- No secrets in code, images or mobile bundles; secrets manager; secret scanning.
- Audit logs for security events without secrets; alerting on anomalies.

- Auth failures fail closed; dependencies' failure modes understood.

Appendix D: Glossary

Term	Meaning
AAA	Authentication, Authorization, Accounting (auditing)
ABAC	Attribute-based access control: decisions from attributes of subject, resource, action, environment
Access token	Short-lived credential presented to an API (resource server)
`acr` / `amr`	OIDC claims describing authentication context class and methods used
Audience (`aud`)	The intended recipient of a token
BFF	Backend-for-Frontend: a server that holds tokens for a browser app
Bearer token	A credential that grants access to whoever holds it
BOLA / IDOR	Broken object level authorization / insecure direct object reference: accessing others' objects by changing IDs
CIAM	Customer identity and access management
Client credentials	OAuth grant for machine-to-machine access
CSRF	Cross-site request forgery: making a victim's browser send authenticated requests
CSP	Content Security Policy: browser policy restricting script sources, limiting XSS
DPoP	Demonstrating Proof of Possession (RFC 9449): binding tokens to a client key
FIDO2 / WebAuthn	Standards behind passkeys and security keys
HMAC	Keyed hash proving integrity and authenticity with a shared secret
ID token	OIDC JWT telling a client who authenticated
JWKS	JSON Web Key Set: published public keys for verifying tokens
JWT	JSON Web Token: signed (sometimes encrypted) JSON claims
mTLS	Mutual TLS: both client and server present certificates
OAuth 2.0 / 2.1	Framework for delegated authorization / its consolidated, hardened revision
OIDC	OpenID Connect: identity layer on OAuth 2.0
PDP / PEP	Policy decision point / policy enforcement point
PKCE	Proof Key for Code Exchange: binds an authorization code to the client instance
Principal	Any entity that can be authenticated and act: user, service, device

Term	Meaning
RBAC	Role-based access control
ReBAC	Relationship-based access control (e.g. Zanzibar, OpenFGA, SpiceDB)
Refresh token	Long-lived credential used only to obtain new access tokens
Relying Party (RP)	The application relying on an identity provider or authenticator
SAML	XML-based federation protocol common in enterprise SSO
SCIM	Standard API for provisioning and deprovisioning users
Scope	Permission granted to an OAuth client
Session fixation	Attack where an attacker sets a victim's session ID before login
SSO	Single sign-on: one authentication used across many applications
Step-up	Requiring stronger or fresher authentication for a sensitive action
TOTP	Time-based one-time password (authenticator apps)
Trust boundary	Line between components with different levels of trust
Workload identity	Platform-issued identity for software workloads without stored secrets
XSS	Cross-site scripting: attacker script running in your page
Zero trust	Security model that verifies every request instead of trusting network location

Appendix E: Further Reading

Primary sources are worth more than blog posts. Read these as you reach the related lessons.

Standards and guidance

- NIST SP 800-63-4 suite, especially **SP 800-63B-4** (authentication, July 2025) and **SP 800-207** (Zero Trust Architecture).
- **RFC 6749** (OAuth 2.0), **RFC 7636** (PKCE), **RFC 8252** (OAuth for Native Apps), **RFC 9700** (OAuth 2.0 Security BCP), **RFC 8628** (Device Grant), **RFC 8693** (Token Exchange), **RFC 9449** (DPoP), **RFC 7662** (Token Introspection), and the OAuth 2.1 and *OAuth 2.0 for Browser-Based Applications* drafts.
- **RFC 7519** (JWT), **RFC 8725** (JWT Best Current Practices), **RFC 7517** (JWK).
- **OpenID Connect Core 1.0** and Discovery; **W3C WebAuthn** Level 2/3; FIDO Alliance passkey resources.
- **RFC 7643/7644** (SCIM 2.0); OASIS SAML 2.0 specifications.

OWASP

- OWASP Top 10:2025 and API Security Top 10 (2023).
- OWASP Cheat Sheet Series: Authentication, Password Storage, Session Management, CSRF Prevention, JSON Web Token, OAuth 2.0, Authorization, Multifactor Authentication.
- OWASP Application Security Verification Standard (ASVS), chapters on authentication, session management and access control. Excellent as a requirements checklist.
- OWASP Mobile Application Security (MASVS/MASTG) for React Native apps.

Papers and engineering writing

- *Zanzibar: Google's Consistent, Global Authorization System* (USENIX ATC 2019).
- Google's BeyondCorp paper series (published in ;login: and on Google research pages).
- Netflix Tech Blog posts on edge authentication and identity propagation.
- Public post-incident reports from companies such as Okta, Uber, Cloudflare and GitHub.
- Adam Shostack, *Threat Modeling: Designing for Security*.

Tools to learn

Burp Suite Community, OWASP ZAP, mitmproxy, Proxyman or Charles, jwt.io, Keycloak (to see a full IdP locally), OpenFGA playground, OPA playground, Cedar playground, SimpleWebAuthn examples, and the `jose` and `openid-client` libraries.

> **FINAL NOTE**
>
> **Standards evolve and libraries come and go. The principles in this book — trust boundaries, evidence and confidence, per-request authorization, least privilege, short-lived and revocable credentials, defense in depth, and failing closed — will outlast every specific tool named here.**